

DATA MINING:

Indice

1	Introduzione	1
1.1	The Big Data Explosion	1
1.1.1	Cosa è il Big Data	1
1.2	Il Data Mining	2
1.3	Concept Learning	2
1.3.1	Alcuni esempi	2
1.3.2	Gli algoritmi del Concept Learning	4
1.4	Il Data Mining è un task induttivo	5
1.4.1	Induzione vs Deduzione	5
1.5	Gli altri Task del DM	5
1.6	Data Mining come supporto alle decisioni	5
1.7	Il processo KDD	5
2	Concept Learning	6
2.1	Il Training set	6
2.2	Il concetto	8
2.2.1	La Classificazione	8
2.3	Capacità di Generalizzazione e Spazio delle Ipotesi	9
2.4	Definizione del problema	10
2.5	Concept Learning come un problema di ricerca	10
2.5.1	Naive approach	11
2.6	Ordinare lo spazio delle ipotesi	11
2.7	Find-S algorithm	12
2.7.1	An Examples	14
2.8	Version space e l'algoritmo "candidate-elimination"	16
2.9	Le ipotesi in DNF	26
2.9.1	Utilizzate "Candidate Elimination" per indurre ipotesi DNF	26
2.9.2	Le ipotesi approssimate	27
2.9.3	Overfitting (sovradattamento)	27
2.10	Classificazione- Valutazione del modello	29
2.10.1	Testing del classificatore	29
2.10.2	Tecniche per la valutazione delle performance	32
2.10.3	Holdout	32
2.11	Cross Validation: K-fold-cross-validation:	32
3	Entropy come misura del disordine	35
3.1	L'entropia secondo la teoria dell'informazione	35
3.2	Entropia nel Data Mining	36
4	Decision tree	37
4.1	DTs il problema del learning	39
4.2	L'idea di base Top-Down dei decision tree	39
4.3	Algoritmo ID3	40
4.4	L'algoritmo ID3 base	41
4.5	L'Information Gain	42
4.6	L'indice GINI	44
4.7	Attributi continui	44

4.7.1	Overfitting	44
4.8	Ridurre la dimensione di un DT	44
4.9	Estensioni di ID3	45

1 Introduzione

In questi ultimi anni stiamo assistendo ad un evento di grande impatto sulle dinamiche della società moderna. Esso viene definito come *The Big Data Explosion*.

1.1 The Big Data Explosion

Oggi i dati vengono considerati il nuovo oro e essi vengono prodotti in quantità enormi ogni momento grazie alla grandissima diffusione di internet; oggi tutto gira intorno alla rete anche le operazioni più semplici. Infatti, per esempio, ogni acquisto che noi facciamo viene doverosamente memorizzato, ogni transazione monetaria viene accuratamente registrata, ogni click effettuato sul web finisce all'interno di un archivio appositamente creato per tenere la loro traccia etc. etc..

In questo caso è importante prendere in considerazione numerosi strumenti che contribuiscono ad aumentare notevolmente la quantità di dati presenti:

1. *I social networks*: Blogs, Forum, news ecc generano un enorme ammontare di dati in forma testuale i quali sono tutti completamente reperibili in rete;
2. *The Internet of Things (IoT)*: permette agli oggetti, come veicoli, sensori, smartphone ecc di collezionare e scambiare dati attraverso la rete. Per esempio i vagoni di un treno sono equipaggiati con centinaia di sensori i quali generano dati ad una frequenza di *ms*;

Tutti questi dati vengono utilizzati in diversi ambiti dal campo medico a quello dell'intelligenza artificiale. Infatti, ci sono per esempio dei dispositivi indossabili e delle App per la salute che fanno parte delle *IoT* mediche che consentono di monitorare e favorire la medicina preventiva.

Tutta questa quantità di dati è utile anche alla *prognostica* una disciplina che di occupa di predire dei fallimenti che stanno per verificarsi all'interno di un sistema o componente (di qualsiasi abito).

Noi oggi viviamo in un mondo *interconnesso* dove i dati sono reperibili come mai prima! Questa è l'era del **Big Data**!

1.1.1 Cosa è il Big Data

I *Big Data* sono dei set di dati estremamente grandi che possono essere analizzati per individuare *pattern*, *tendenze* e *associazioni* specialmente riguardanti i comportamenti e interazioni umane.

Il Big Data può essere definito attraverso le famose **3V**:

- **Volume**: cioè l'enorme quantità di dati reperibili;
- **Varietà**: cioè l'enorme numero di tipi diversi di dato reperibili;
- **Velocità**: cioè l'enorme velocità con il quale vengono generati nuovi dati ogni momento.

1.2 Il Data Mining

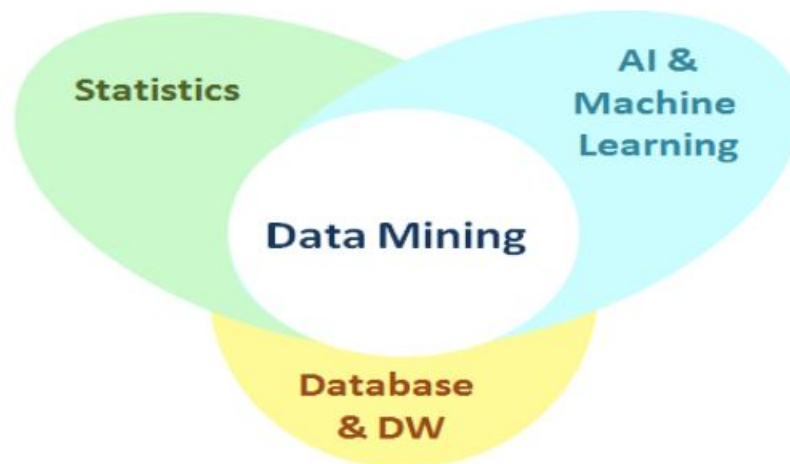
Il grande ammontare di dati che possono essere ottenuti non sono solamente dati statici bensì contengono al loro "interno" una *conoscenza* che è nascosta e implicita in essi. Per tale motivo c'è bisogno di utilizzare delle tecniche per estrapolare queste importanti informazioni in modo da utilizzarli per diversi scopi; è *proprio questo il compito del Data Mining*.

Il Data Miner è, quindi, una persona che estrae la conoscenza dai dati.

*Il Data Mining (DM) è l'insieme di tutte le tecnologie (come algoritmi, metodi, tools ecc) che consentono l'estrazione delle **informazioni nascoste e interessanti** (come regole, regolarità, pattern, vincoli ecc) da un grande ammontare di dati che hanno tipi e formati differenti in modo da trovare **correlazioni** (o **pattern**) all'interno di tali dati.*

Il Data Mining si basa su delle forti fondamenta teoriche/matematiche come:

- Intelligenza artificiale: Machine Learning e Logiche
- Statistiche
- Database Management System



1.3 Concept Learning

Il Concept Learning è una delle cose principali di cui si occupa il DM e ha l'obiettivo di trovare dei *modelli* generali da esempi pratici e reali specifici. Esso è un task induttivo volto ad imparare un concetto da un insieme di esempi.

1.3.1 Alcuni esempi

Supponendo di avere i seguenti esempi memorizzati nel nostro database: L'*attributo target* è l'attributo che si vuole determinare in base ai valori degli altri attributi presenti all'interno dell'entità contenente i diversi esempi.

Id	Body temp	Aquatic	Aerial	Legs	Hibernates	Gives birth	Target attribute
human	Warm	No	No	Yes	No	Yes	mammal
Python	Cool	No	No	No	Yes	No	reptile
salmon	Cool	Yes	No	No	No	No	fish
Whale	Cool	Yes	No	No	No	Yes	mammal

Nel nostro caso si vuole imparare il concetto di "mammifero" cioè quali sono le proprietà che caratterizzano un mammifero. Nel db sono presenti sia degli esempi *positivi* (cioè che hanno il valore mammal nell'attributo target) sia *negativi*.

Una possibile ipotesi potrebbe essere "Mammal if aerial=no and hibernates=no and gives birth=yes".

Secondo i nostri dati quindi un mammifero è un essere vivente che non è volatile, che non va in letargo e che fa dei figli (gli altri attributi non sono importanti). Questa ipotesi è consistente col nostro training set di dati.

*un'ipotesi è **consistente** se per ogni istanza nel training set essa soddisfa solamente gli esempi positivi e mai quelli negativi.*

Il modello per il concetto mammifero può essere visto come:

- *un tool descrittivo* cioè un tool che dimostra quali sono le caratteristiche di un mammifero (utile per biologi, zoologi..)
- *un tool predittivo* cioè che può essere usato per predire l'etichetta di un nuovo essere vivente.

La task di predizione dell'attributo target è detto *Classificazione*

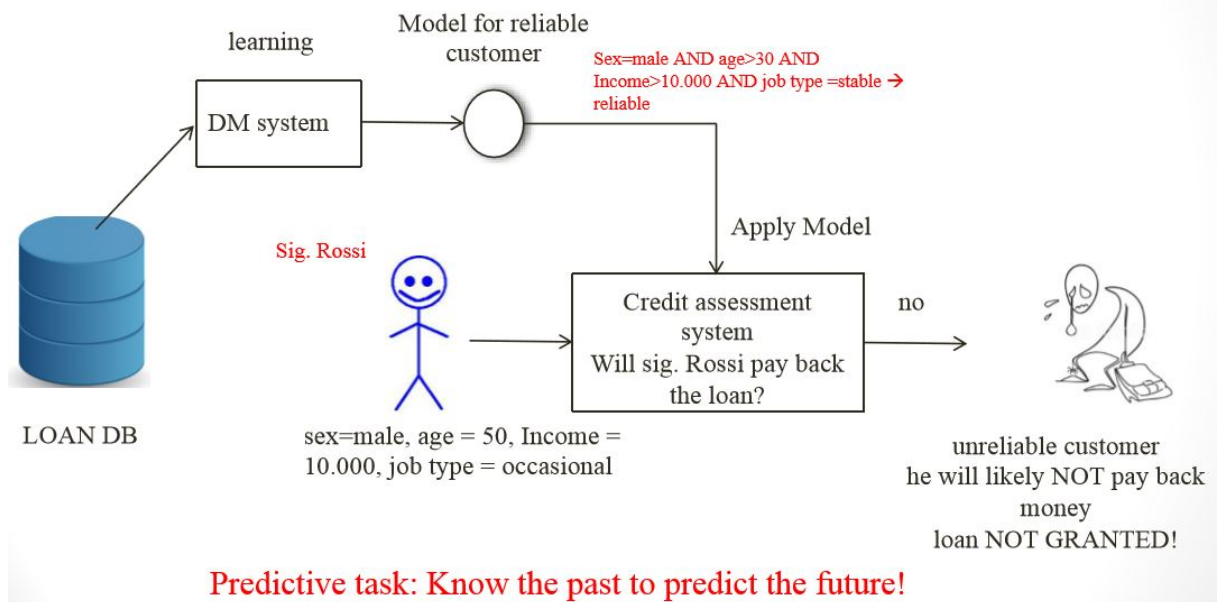
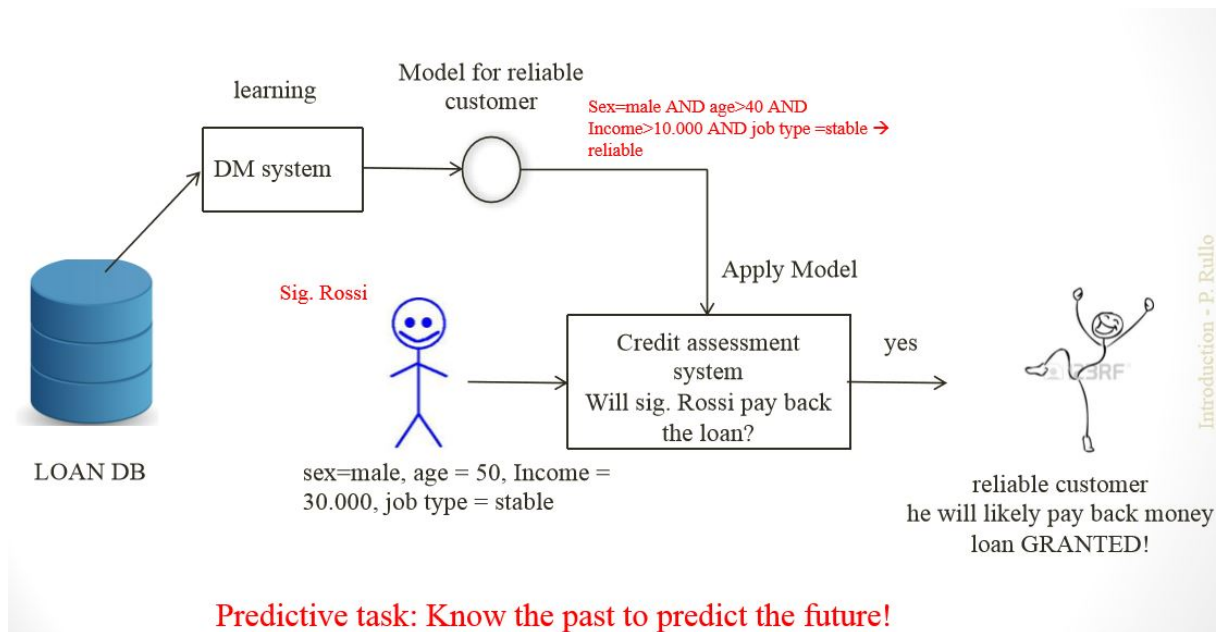
Un altro importante esempio può essere quello della valutazione del rischio di Credito che concedono le banche.

Ogni banca possiede un proprio database dove memorizza tutte le informazioni dei crediti concessi e sulle relative operazioni effettuate in passato. Quando succede che un nuovo cliente chiede un credito essa deve prendere una decisione in merito, per fare ciò è possibile "imparare" dai dati posseduti nel database in modo da prendere la decisione migliore poiché i dati non mentono!

In questo caso la fase di Concept learning studia dai dati storici il concetto di "Cliente Affidabile" cioè una descrizione delle proprietà che un cliente dovrebbe avere in modo da essere considerato affidabile dal punto di vista bancario.

Una volta trovato tale modello è possibile utilizzarlo per predire se, ogni qual volta un nuovo cliente chiede un credito, esso sarà in grado o meno di restituirlo. In questo caso il modello ha anche una capacità descrittiva

Di seguito due possibili applicazioni di tale modello:



1.3.2 Gli algoritmi del Concept Learning

Un *learning algorithm* è formato sui dati degli esempi disponibili (sul database) per imparare un *concetto* (attributo target). Il concetto imparato (o modello) è quindi usato a scopi descrittivi oppure per classificare delle nuove istanze sconosciute (per una predizione quindi).

In pratica, il modello dedotto è la conoscenza che abbiamo estratto dai dati che descrivono le esperienze passate

*Il Concept Learning (e in generale il DM) è un task **induttivo**.*

1.4 Il Data Mining è un task induttivo

Il DM è un task induttivo poiché estrae teorie generali dai dati osservati.

I metodi di learning puramente induttivi formulano ipotesi generali trovando regolarità empiriche sugli esempi. (Tom M. Mitchell, 1997, p. 334)

1.4.1 Induzione vs Deduzione

L'*induzione* parte dal particolare per arrivare al generale mentre al contrario la *deduzione* parte dal generale per arrivare al particolare.

L'induzione è tipica delle scienze sperimentali (come fisica, biologia ecc) e viene fatta estraendo delle teorie generali dai dati osservati. Vengono, infatti, osservati un insieme di esempi, estrapolati dei pattern, create delle generalizzazioni e infine individuato un modello o teoria. Tuttavia, tale modello indotto da un insieme di esempi può essere errato nonostante gli esempi sono corretti - conoscenza probabilistica.

La deduzione è, invece, tipica della matematica vengono dedotte infatti delle conseguenze da delle teorie generali. Se la teoria (premessa) è valida allora la conseguenza è vera - ragionamento deduttivo è una forma base di ragionamento valido.

1.5 Gli altri Task del DM

- *Mining di Clusters*: un cluster è un gruppo di oggetti simili. L'analisi dei cluster si riferisce alla formazione di gruppi di oggetti molto simili tra loro ma molto diversi dagli oggetti di altri cluster
- *Mining di associazioni*: si riferisce al processo di scoperta delle relazioni all'interno dei dati e alla determinazione delle regole di associazione.
- *Analisi degli Outlier*: cioè le analisi dei valori anomali. I valori anomali possono essere definiti come oggetti dato anomali, cioè "distanti" da altri oggetti.

1.6 Data Mining come supporto alle decisioni

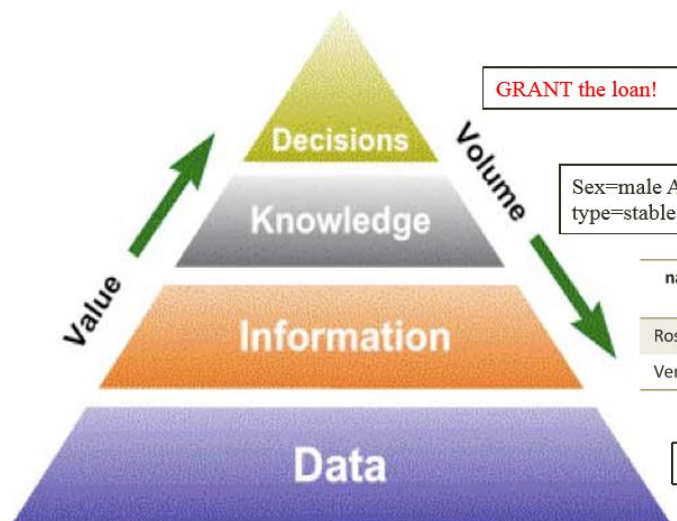
Il Data mining aiuta le persone ad utilizzare le informazioni disponibili per ottenere della conoscenza riguardo al dominio di interesse e prendere delle decisioni informate.

1.7 Il processo KDD

Il processo totale di estrazione della conoscenza utile da grandi database sono chiamati *Knowledge discovery in database* (KDD).

Le sue fasi sono:

- Acquisizione dei dati
- Pre-processing dei dati
- Data mining (induzione di un modello)
- Valutazione e visualizzazione del modello



2 Concept Learning

Lo scopo principale del concept learning è quello di acquisire dei concetti generali da training set di esempi specifici. Quindi, dato un insieme di esempi, relativi ad un determinato dominio di applicazione, noi vogliamo imparare un concetto generale, chiamato *modello* basandoci sui valori contenuti in questo insieme.

Il concept learning, da un lato, può essere visto come un'attività di ricerca di una correlazione fra i valori degli attributi oppure, da un altro lato, come un'attività di ricerca dell'attributo target.

Come detto precedentemente, il modello derivato può essere utilizzato sia a scopi *descrittivi* che *predittivi*.

2.1 Il Training set

Un **training set** S è un insieme di esempi, cioè istanze utilizzate come base per imparare un concetto.

Ogni esempio nel training set:

- è associato ad una etichetta (label) o classe;
- è caratterizzato da un insieme di attributi, cioè proprietà che lo descrivono

L'associazione che intercorre fra esempi e class label è chiamata **funzione target**.

Detto ciò, il training set può essere definito meglio come:

$$S = \langle e, f(e) \rangle$$

dove f è la funzione target che mappa ogni esempio ad un valore della classe target.

Quindi, S è l'insieme delle tuple composte da un esempio e la relativa etichetta.

Un'*ipotesi* può essere semplicemente rappresentata come una formula proposizionale definita sugli attributi. Essa è in Conjunctive Normal Form. In particolare,

essa può essere vista come un vettore che specifica il valore per ogni attributo o meglio vincola il valore che esso può assumere. Ogni attributo potrebbe avere come vincolo:

- ? per indicare che qualsiasi valore per quello attributo è; accettabile nell'ipotesi
- **un valore specifico** per l'attributo;
- $\emptyset$ quando nessun valore è accettato per un attributo.

Un ipotesi potrebbe quindi avere la seguente forma:

$$h = \langle ?, ?, value, value, ? \rangle$$

cioè un vettore di vincoli sugli attributi.

Un ipotesi per essere un modello deve essere **consistente** col training set. Ciò avviene se e solo se h restituisce il concetto che si sta imparando per ogni esempio positivo all'interno del TS ma per nessuno degli esempi negativi.

NB: più un training set è grande e staticamente significativo più è raggiungibile un buon modello su esso. I concetti infatti non possono essere esattamente imparati da dati limitati ma solamente approssimati. Perciò la conoscenza induttiva è probabilistica

L'ipotesi dell'apprendimento induttivo: *Qualsiasi ipotesi trovata per approssimare bene la funzione target su un set sufficientemente grande di training examples approssimerà bene anche la funzione target su altri esempi non osservati.*

Esempio

The mammal training set

Id	Body temp	Aquatic	Aerial	Legs	Hibernates	Gives birth	Class
human	Warm	No	No	Yes	No	Yes	mammal
Python	Cool	No	No	No	Yes	No	reptile
salmon	Cool	Yes	No	No	No	No	fish
Whale	Cool	Yes	No	No	No	Yes	mammal

In questo esempio una possibile ipotesi potrebbe essere:

$$h : bodytemp = ? \wedge aquatic = ? \wedge aerial = no \wedge legs = ? \wedge hibernates = no \wedge givebirth = yes$$

che diventa nella notazione spiegata prima:

$$\langle ?, ?, no?, no, yes \rangle$$

secondo questa ipotesi un "mammifero è un essere vivente il quale non è un volatile, non va in letargo e fa dei figli".

Bisogna però determinare se questa sia consistente col TS .

In questo caso sia "humans" che "whales" la soddisfano, quindi, $h(human) = mammal$ e $h(whale) = mammal$. Se entrambi sono mammiferi in accordo con h allora h è consistente con gli esempi positivi, cioè $h(e) = f(e)$, for each e in human, whale.

Mentre né "Pythons" né "Salmon" soddisfano tale formula quindi $h(python) \neq mammal$ e $h(salmon) \neq mammal$ e ciò vuol dire che non sono mammiferi secondo questa ipotesi. Per questo motivo h è consistente con gli esempi negativi.

Dato che questa ipotesi è consistente con tutti gli esempi del training set essa è un modello

Supponiamo adesso che arrivi una nuova istanza che descrive una tartaruga nella forma: $\langle cool, yes, no, yes, no, no \rangle$ che non esiste nel nostro training set. Per definire se essa sia un mammifero o meno bisogna confrontarla col modello precedentemente trovato $\backslash mammal \rightarrow h = \langle ?, ?, no, ?, no, yes \rangle$. Dato che la tartaruga non soddisfa h allora la risposta è *NO* (in accordo col modello di mammifero)

2.2 Il concetto

Un *concetto* è un sottoinsieme di oggetti definiti su un insieme di dati oppure, in alternativa, un concetto è una espressione proposizionale definita sull'insieme dei dati.

Per esempio, nella prima considerazione possiamo pensare al concetto di mammifero che altro non è che un sottoinsieme di tutti gli oggetti (cioè l'insieme degli esseri viventi) che sono *etichettati* come mammiferi. Mentre, nella seconda considerazione si può pensare ad una formula definita su tutti gli esseri viventi il quale valore è vero per i mammiferi e falso per tutti gli altri esempi.

Dato un insieme di esempi *etichettati* come *membri* oppure *non membri* di un determinato concetto C , il "Concept Learning" consiste nel dedurre automaticamente la definizione generale di tale concetto C .

2.2.1 La Classificazione

Una volta che abbiamo imparato un concetto, possiamo usare il modello per **classificare** le nuove istanze.

Quindi, la **Classificazione** è l'attività che assegna una class label ad un'istanza basandosi sui valori degli attributi di tale istanza. Da questo, si può capire facilmente che la classificazione è un'attività *predittiva* poiché essa consente di predire l'etichetta di una nuova istanza basandosi sulle istanze (o esempi) passate.

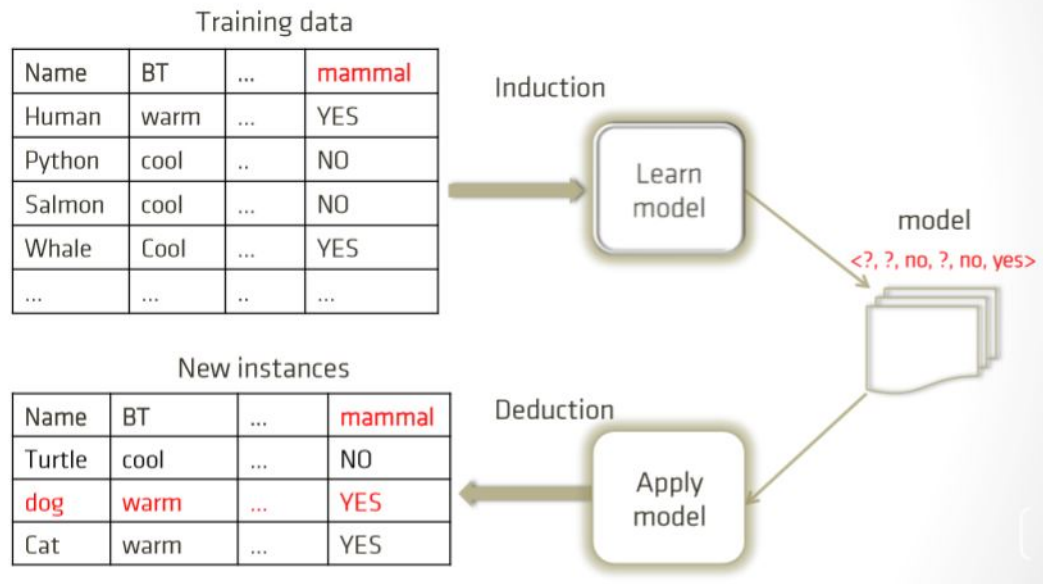
La classificazione è spesso usata come sinonimo di Concept Learning.

Quindi come si può osservare dalla figura gli step sono:

1. Imparare un concetto per *induzione* a partire da un *Training data*;
2. Costruire un modello;

3. Applicare il modello precedentemente trovato sulle *nuove istanze* al fine di *Dedurre* la loro class label;

Classification



2.3 Capacità di Generalizzazione e Spazio delle Ipotesi

Come detto prima l'assunzione fondamentale che viene fatta nell' *inductive learning* è che la migliore ipotesi riguardante le istanze non viste è quell'ipotesi che meglio si adatta alle istanze osservate nel training data. So, un buon modello è uno con un'alta capacità di generalizzazione e che quindi è capace di assegnare l'etichetta corretta alle nuove e mai viste istanze.

Lo *Spazio delle Ipotesi* è l'insieme di tutte le possibili ipotesi sui valori degli attributi, cioè l'insieme di tutte le formule proposizionali che possono essere espresse in un dato linguaggio.

Quindi, come sappiamo un'ipotesi è un'espressione proposizionale definita sugli attributi di un training set il quale descrive un insieme di possibili esempi mentre un *Linguaggio delle ipotesi* è il tipo di espressioni proposizionali usate per rappresentare delle ipotesi.

Lo spazio delle ipotesi è l'insieme di tutte le possibili congiunzioni sugli attributi di un training set.

Data una congiunzione $h = \langle h_1, \dots, h_n \rangle$ e un esempio $e = \langle e_1, \dots, e_n \rangle$ si dice che e soddisfa h se e "fits into" h , cioè: per ogni $i = 1, \dots, n$, $h_i = ?$ oppure $h_i = e_i$.

- Se h è un'ipotesi per la classe c e e soddisfa h allora $h(e) = c$, altrimenti $h(e) \neq c$.

- h è *consistente* con gli esempi e in S se:
 - $h(e) = c$ e $f(e) = c$
 - $h(e) \neq c$ e $f(e) = c$
- h è *consistente* con S se esso è consistente con qualsiasi esempio in S .
- h è un *modello* per S se esso è consistente con S .

Un problema binario è uno con la classe $c \in 0, 1$ quindi la funzione target è $f : \text{examples} \rightarrow 0, 1$. Per convenzione, in questo caso la classe positiva è sempre 1;

2.4 Definizione del problema

Sia S un insieme di (training) esempi

$$S = \langle e, f(e) \rangle$$

dove,

- e è un esempio $\langle e_0, \dots, e_n \rangle$
- f è la funzione target e $f(e)$ è invece il concetto target (o meglio l'etichetta della classe) dell'esempio e . Il *Problema* è imparare da S un'ipotesi $h \in H$ il quale è un *modello* per S , e cioè che è consistente con gli esempi in S

Se e soddisfa h , allora $h(e) = 1$ altrimenti $h(e) = 0$

2.5 Concept Learning come un problema di ricerca

Il concept learning può essere visto come un'attività di ricerca su un enorme spazio delle ipotesi definito implicitamente dalla rappresentazione delle ipotesi. L'obiettivo di questa ricerca è quello di cercare l'ipotesi che si adatta meglio al training set.

In modo più formale. *Search Problem:* dato uno spazio di ricerca H e un training set $S = \langle e, f(e) \rangle$, trovare $h \in H$ (se esiste) *consistente* con i dati di esempio, cioè tale che : $h(e) = f(e)$ per qualsiasi esempio in S .

H è quindi sono tutte le possibili ipotesi, cioè lo spazio delle ipotesi. All'interno di esso si possono trovare tutte le ipotesi sintatticamente distinte e semanticamente distinte. Il numero delle ipotesi sintatticamente distinte è molto grande, infatti, supponendo di avere 3 attributi nel nostro training set A_1, A_2, A_3 che possono avere rispettivamente 3, 2, 2 valori differenti. In questo caso, abbiamo $5 * 4 * 4 = 80$ istanze sintatticamente distinte. Ovviamente, al numero dei possibili valori per ogni attributo vanno anche sommati i simboli $?$ e $\emptyset$ (ecco perché 5 e 4). Tuttavia, in esse ci sono anche tutte le ipotesi che contengono uno o più $\emptyset$ che rappresenta l'insieme vuoto delle istanze, e classificherà tutte le istanze come negative. Perciò il numero delle ipotesi semanticamente distinte sono solamente $1 + (4 * 3 * 3) = 37$.

Se consideriamo il learning come un problema di ricerca allora è naturale che vengano considerati a tal fine tutti gli algoritmi che hanno differenti strategie per trovare quello che più si adatta al problema. In particolare, quelli più interessanti sono quelli capaci di effettuare una ricerca efficiente su un enorme o infinito spazio delle ipotesi H , in order to find the ipotesi che meglio rappresentano il training set.

In generale, il numero delle ipotesi congiuntive in H è esponenziale rispetto al numero n degli attributi, cioè, $s = m^n$ assumendo che m è il numero dei possibili valori per ogni attributo.

2.5.1 Naive approach

Input: un training set $S = \langle e, f(e) \rangle$ e uno spazio delle ipotesi H

Inizio: Ricerca naive

- foreach training example $\langle e, f(e) \rangle$
 - remove from H any hypothesis h for which $h(e) \neq f(e)$, h in questo caso infatti è inconsistente con S quindi va eliminata
- Output H - (potrebbe essere vuoto) insieme dei modelli

end

Quindi ogni esempio E viene testato con tutte le ipotesi presenti in H . Se una qualche ipotesi non dovrebbe essere consistente rispetto ad E essa viene cancellata. Alla fine di tutto il processo rimangono solamente le ipotesi realmente consistenti col TS e cioè tutti i possibili modelli.

In generale tuttavia, questo approccio è inefficiente perché esso richiede l'enumerazione esaustiva di tutte le possibili ipotesi presenti in H per ogni training example(!). Uno dei possibili rimedi a questo problema potrebbe essere: sfruttare l'ordinamento parziale sulle ipotesi.

2.6 Ordinare lo spazio delle ipotesi

Per risolvere il problema dell'enumerazione di tutto lo spazio delle ipotesi H (infinitely many), molti algoritmi per il Concept Learning organizzano la ricerca su H facendo affidamento su una struttura molto efficiente che esiste per ogni problema di concept learning: **un ordinamento general-to-specific delle ipotesi**. Sfruttando il vantaggio di questa struttura naturale delle occorrenze sullo spazio delle ipotesi H , è possibile progettare algoritmi di learning che cercano esaustivamente sull'infinito spazio delle ipotesi senza enumerare esplicitamente ogni ipotesi al suo interno.

Dato un training set S e le ipotesi h e g su S , $h \geq g$ (si legge come "h è più generale di g" oppure "g è più specifico rispetto a h") se:

$$g(e) = 1 \rightarrow h(e) = 1$$

cioè se è soddisfatta g allora è soddisfatta anche h (perché è più generale). Intuitivamente h copre più esempi rispetto a g .

Esempio:

- $g = \langle \text{cool, yes, ?, ?, no, no} \rangle$, $h = \langle \text{cool, yes, ?, ?, no, ?} \rangle$
- g does satisfy salmon
- h does satisfy both salmon and whale
- Hence, h is more general than g

Id	Body temp	Aquatic	Aerial	Legs	Hibernates	Gives birth	Mammal
human	Warm	No	No	Yes	No	Yes	yes
Python	Cool	No	No	No	Yes	No	no
salmon	Cool	Yes	No	No	No	No	no
Whale	Cool	Yes	No	No	No	Yes	yes

Sintatticamente:

$$h \geq g \text{ con } h = \langle a_1, \dots, a_n \rangle \text{ e } g = \langle b_1, \dots, b_n \rangle$$

se: per ogni $i = 1..n$, $a_i = ?$ oppure $a_i = b_i$. se h impone meno constraints, sugli attributi, rispetto a g essa classifica come positive più istanze rispetto a g . Quindi, essa è più generale di g .

- $g = \langle ?, ?, ?, ?, ?, ? \rangle$ è l'ipotesi più generale, infatti secondo essa sono tutti valori non importanti nella decisione della class label, quindi, essa è soddisfatta da qualsiasi esempio ognuno dei quali viene classificato come un'istanza positiva.
- $s = \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$ è invece l'ipotesi più specifica, indica che nessun valore è consentito di conseguenza essa non è soddisfatta da nessun esempio e quindi essa classifica tutte le istanze come negative.

L'ordinamento "more general than" che può essere indicato con $(H, \geq)$ è un *ordine parziale* (ha le proprietà riflessiva, asimmetrica e transitiva).

2.7 Find-S algorithm

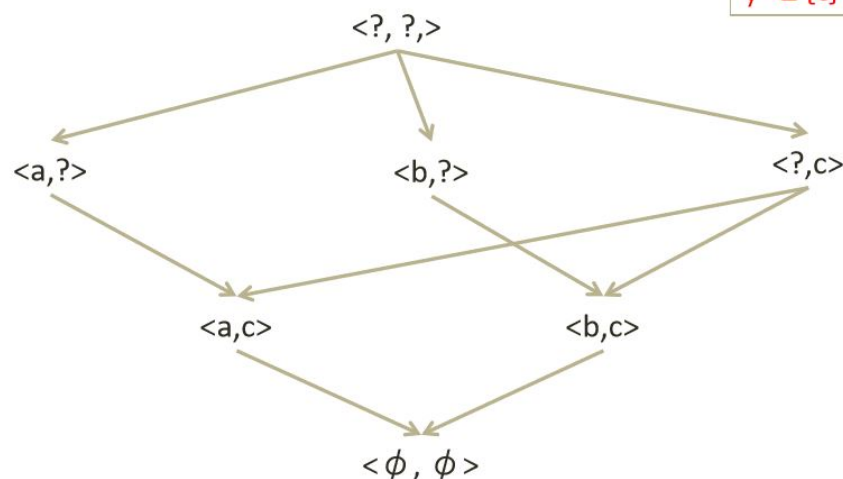
l'ordinamento parziale *more general than* può essere utilizzato per organizzare la ricerca di un'ipotesi consistente col training set osservato. L'idea di base, infatti, è

quella di partire dall'ipotesi H più specifica possibile e successivamente generalizzarla ogni volta che essa non riesce a coprire un'istanza positiva all'interno del training set.

Ogni volta che l'ipotesi viene generalizzata tale generalizzazione è *minima*. Dobbiamo quindi distinguere:

- *Generalizzazione Minima*: viene sostituito un valore di attributo con "?". Per esempio $h = \langle a, b, c, d, ?, ? \rangle$ diventa $h = \langle a, b, ?, d, ?, ? \rangle$
- *Specializzazione Minima*: viene sostituito un "?" con un valore specifico. Per esempio $h = \langle a, b, ?, d, ?, ? \rangle$ diventa $h = \langle a, b, i, d, ?, ? \rangle$

The hypotheses space has a structure



$h = \langle x, y \rangle$
 $x \in \{a, b\}$
 $y \in \{c\}$

Funzionamento dell'algoritmo

Iniziando dall'ipotesi *più specifica* esso esamina *tutte* gli esempi positivi all'interno del training set, uno alla volta. Ad ogni step, se l'ipotesi corrente non dovesse risultare consistente con l'esempio esaminato, esso cerca la successiva ipotesi più generale (attraverso una generalizzazione minima) che è consistente con l'esempio corrente (e tutti i precedenti ovviamente). Dopo aver esaminato tutti gli esempi positivi esso restituisce l'ipotesi h *più specifica* che risulta essere consistente con tutti gli esempi positivi. questa h potrebbe anche coincidere con l'ipotesi più generale in assoluto (quella fatta tutta di ?).

Se h non copre *nessun* esempio negativo allora h è un modello altrimenti vuol dire che *non esiste nessun modello conjunctive*.

Più formalmente:

Input: the training set S ; the space $(H, \geq)$ of all possible conjunctions over the examples in S

begin

- $h = \langle \phi, \phi, \dots, \phi \rangle$
- for each positive example e in S
 - If e does not satisfy h then replace h by the next most specific hypothesis in H according to $\geq$ that is satisfied by e
- if there is any negative example satisfying h then
 - *Output* "no conjunctive model exists"
- else
 - *Output* h is a model

end

2.7.1 An Examples

Training Examples for EnjoySport

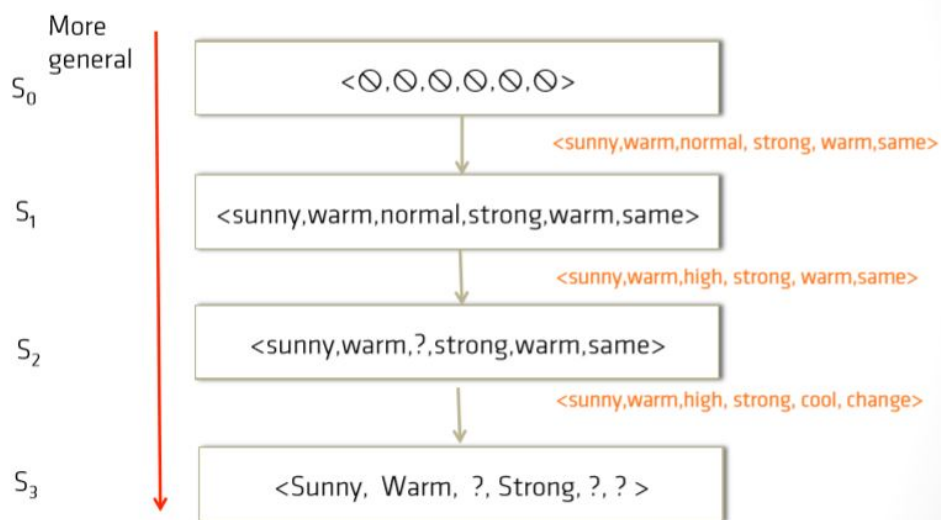
(Mitchell, Machine Learning)

Sky	Temp	Humid	Wind	Water	Forecast	EnjoySpt
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

- Days on which Aldo enjoys his favorite water sport
- Task: given the above training set, find a model to predict the value of "Enjoy Sport" for an arbitrary day based on the values of the other attributes

Find-S Algorithm

the EnjoySport example



- Starting from $\langle \phi, \dots, \phi \rangle$, FindS progressively generalizes the current hypothesis to make it consistent with all positive examples so far examined
- Eventually Find-S outputs
 - $h = \langle \text{Sunny, Warm, ?, Strong, ?, ?} \rangle$
- h is a **model** for the EnjoySport dataset, as it does not cover any negative examples
- h is the **most specific** conjunctive model

Find-S Algorithm

Sky	Temp	Humid	Wind	Water	Forecst	EnjoySpt
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	Very high	Strong	Warm	Same	No

- By considering the positive examples FindS generates
 - $h = \langle \text{sunny, warm, ?, strong, warm, same} \rangle$
- h not a model – satisfies one negative example (the **red one**)

Alcune problematiche: Sebbene FIND-S trovi un'ipotesi coerente con i dati del training set, non ha modo di determinare se ha trovato l'*unica* ipotesi in H coerente con i dati (cioè il concetto obiettivo corretto), o se ci sono molte altre ipotesi coerenti allo stesso modo.

Nel caso in cui ci siano diverse ipotesi consistenti col training set, *Find-S* troverà quello più specifica possibile. Non è chiaro però se dovremmo preferire questa rispetto a quella più generale oppure ad una che ha una generalità intermedia.

In molti problemi pratici di learning potrebbe esserci la possibilità che qualche esempio all'interno del nostro set contenga almeno qualche errore o rumore. Qualche inconsistenza all'interno del training set potrebbe portare questo algoritmo a sbagliare gravemente; questo a causa del fatto che esso ignora gli esempi negativi.

Un altro caso particolare potrebbe verificarsi quando ci sono più ipotesi che sono le più specifiche possibili (quindi hanno lo stesso grado di specializzazione). In questo caso, Find-S deve essere esteso in modo da poter effettuare una backtrack sulle proprie scelte su come generalizzare le ipotesi, questo per prevedere la possibilità che il concetto target potrebbe trovarsi lungo un ramo differente da quello scelto

2.8 Version space e l'algoritmo "candidate-elimination"

L'idea chiave dell'algoritmo **Candidate-Elimination** è quella di restituire una descrizione dell'insieme di *tutte le possibili ipotesi consistenti con gli esempi del training set*. Sorprendentemente, questo algoritmo calcola tale descrizione senza enumerare esplicitamente gli elementi all'interno dell'insieme H . Questo è possibile anche in questo caso grazie all'uso dell'ordinamento parziale *more general than*, questa volta per mantenere una rappresentazione compatta dell'insieme delle ipotesi consistenti e per raffinare in modo incrementale questa rappresentazione ogni volta che una nuova istanza viene incontrata. Tuttavia, le applicazioni pratiche di questi due algoritmi sono limitate a causa della loro debolezza nel lavorare con dei dati sporchi.

L'algoritmo **Candidate-Elimination** cerca tutte le ipotesi descrivibili che sono consistenti con gli esempi osservate.

Definizione: Un ipotesi h è **consistente** con un training set D se e solo se $h(x) = c(x)$ per ogni esempio $\langle x, c(x) \rangle$ in D .

$$\text{Consistent}(h, D) \equiv (\forall \langle x, c(x) \rangle \in D) h(x) = c(x)$$

differenza fra consistente e soddisfa: si dice che un esempio x soddisfa un ipotesi h quando $h(x) = 1$ indipendentemente dal fatto che x sia un esempio positivo o negativo del concetto obiettivo. Tuttavia, se un tale esempio è coerente con h dipende dal concetto obiettivo, e in particolare se $h(x) = c(x)$.

Come detto, questo algoritmo rappresenta l'insieme di tutte le ipotesi consistenti col training set osservato. Tale subset è chiamato *version space* rispetto allo spazio delle ipotesi H e al training example D , perchè esso contiene tutte le versioni plausibili del concetto target.

Definizione: Il **version space**, denotato con $VS_{H,D}$, rispetto allo spazio delle ipotesi H e al training set D , è il sottoinsieme di ipotesi di H consistenti col training example D .

$$VS_{H,D} \equiv \{h \in H \mid \text{Consistent}(h, D)\}$$

Representing Version Spaces

- **Specific boundary** S : most specific hypothesis in VS
- **General boundary** G : set of most general hypotheses in VS



$S: \{h_0\}$
 $G: \{h_4, h_5\}$

Sky	Temp	Humid	Wind	Water	Forecst	EnjoySpt
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Candidate Elimination algorithm

- The algorithm starts with
 - Specific boundary $S = \{<\emptyset, \emptyset, \dots, \emptyset>\}$
 - General boundary $G = \{<?, ?, \dots, ?>\}$
 - For each example e in the training set the hypotheses in S and G are modified in order to make them consistent with e
 - In general
 - hypotheses in G are specialized
 - hypotheses in S are generalized
- so restricting the version space to eliminate hypotheses not consistent with e

Candidate-elimination the EnjoySport example

S_0

$<\emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset>$

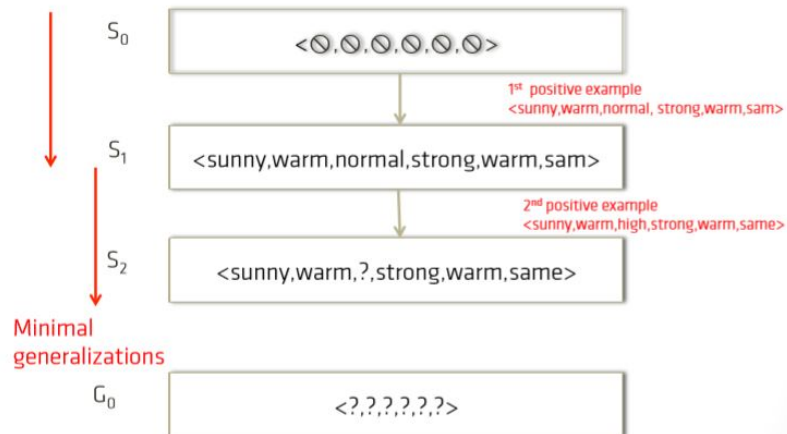
1st positive example
 $e = \langle \text{sunny, warm, normal, strong, warm, sam} \rangle$

Is e consistent with S ? NO - generalize

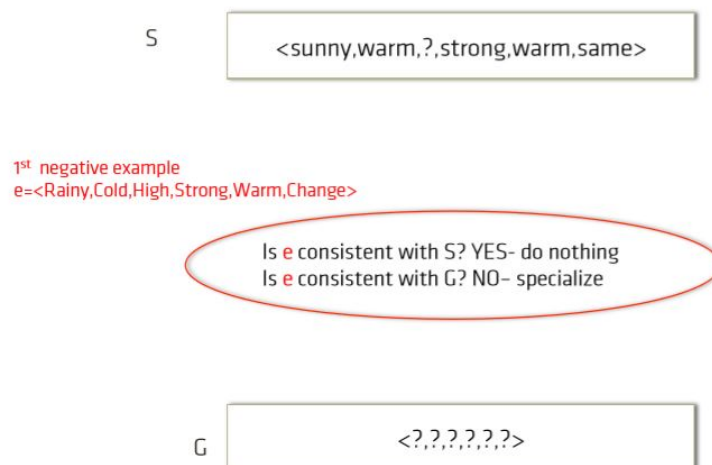
G_0

$<?, ?, ?, ?, ?, ?>$

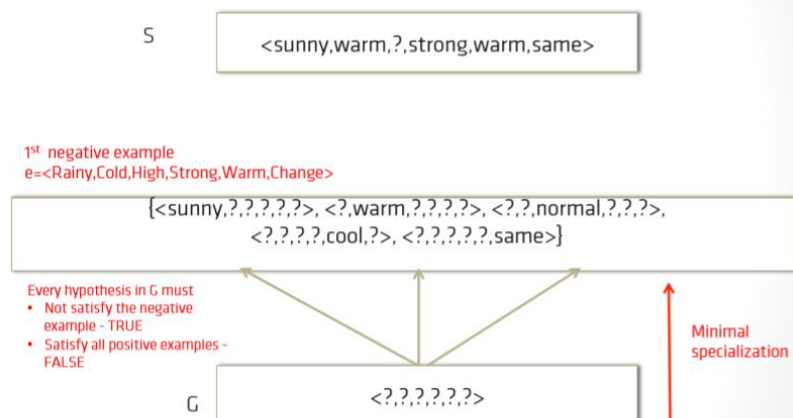
Candidate-elimination the EnjoySport example



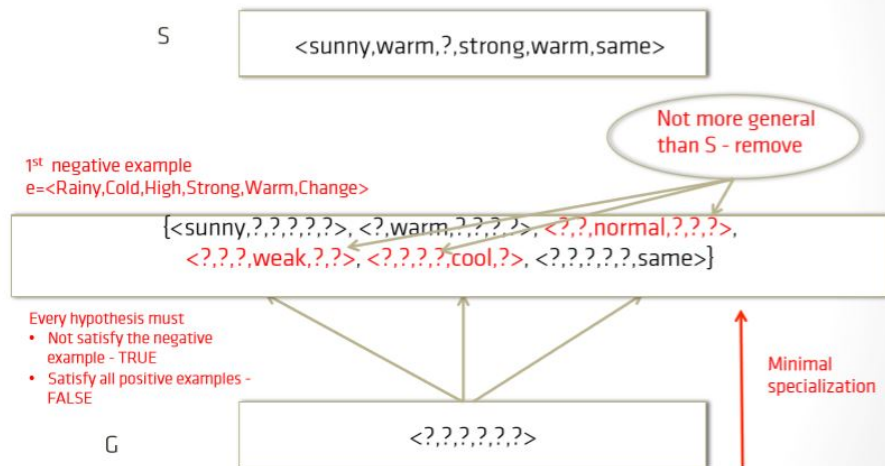
Candidate-elimination the EnjoySport example



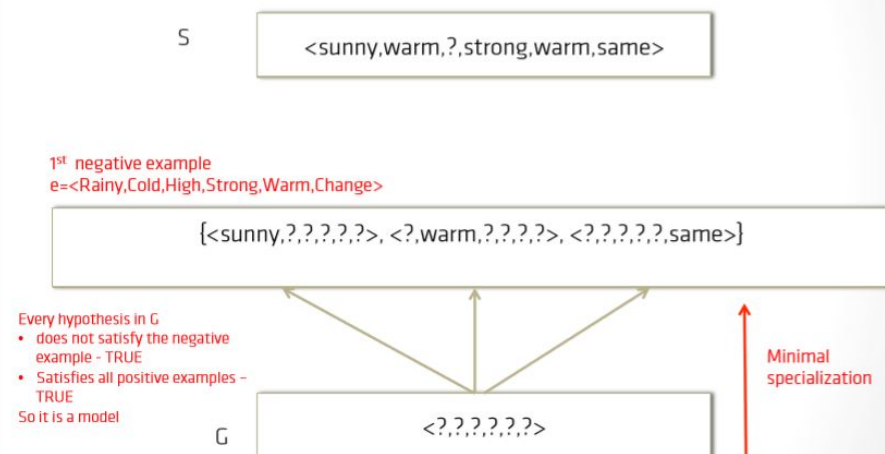
Candidate-elimination the EnjoySport example



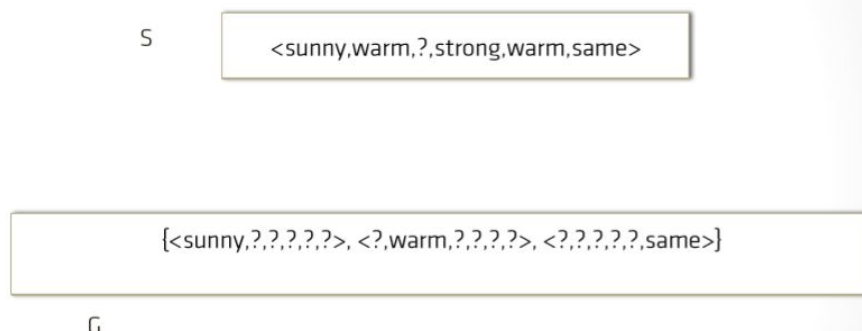
Candidate-elimination the EnjoySport example



Candidate-elimination the EnjoySport example



Candidate-elimination the EnjoySport example



Candidate-elimination the EnjoySport example

S

<sunny,warm,?,strong,warm,same>

3dr positive example
e=<Sunny,Warm,High,Strong,Cool,Change>

is e consistent with S? NO - generalize

{<sunny,?,?,?,?,?>, <?,warm,?,?,?,?,?>, <?,?,?,?,?,same>}

G

Candidate-elimination the EnjoySport example

S

<sunny,warm,?,strong,?,?>

3dr positive example
e=<Sunny,Warm,High,Strong,Cool,Change>

{<sunny,?,?,?,?,?>, <?,warm,?,?,?,?,?>, <?,?,?,?,?,same>}

G

Candidate-elimination the EnjoySport example

S

<sunny,warm,?,strong,?,?>

3dr positive example
e=<Sunny,Warm,High,Strong,Cool,Change>

not more general than s
- remove

{<sunny,?,?,?,?,?>, <?,warm,?,?,?,?,?>, <?,?,?,?,?,same>}

G

Candidate-elimination the EnjoySport example

S

<sunny,warm,?,strong,?,?>

No more examples

{<sunny,?,?,?,?,?>, <?,warm,?,?,?,?,?>}

G

Candidate-elimination the EnjoySport example

S

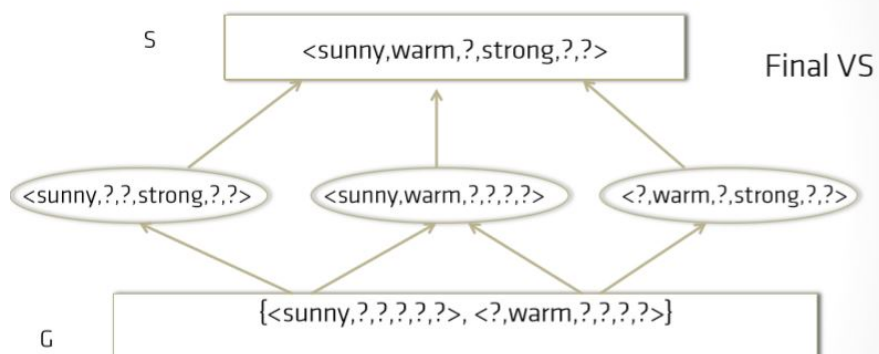
<sunny,warm,?,strong,?,?>

S is the Specific boundary and G the General boundary of the version space

{<sunny,?,?,?,?,?>, <?,warm,?,?,?,?,?>}

G

Candidate-elimination the EnjoySport example



All possible hypotheses compatible with the examples

Il teorema del VS

- Il **Limite Generale** G , rispetto ad uno spazio delle ipotesi H e ad un training Set D , è l'insieme dei membri *più generali* in assoluto di H consistenti con D .
- Il **Limite Specifico** S , rispetto ad uno spazio delle ipotesi H e ad un training Set D , è l'insieme dei membri *meno generali* in assoluto di H consistenti con D .

Basandosi su queste due definizioni il Teorema del Version Space afferma che: *ogni ipotesi che si trova fra S e G è consistente cioè è un modello.*

Supponiamo di avere un ipotesi h , una ipotesi più generale g e un ipotesi meno generale s tali per cui $g \geq h \geq s$.

- $h \geq s$ vuol dire che ogni esempio (positivo) classificato da s , viene classificato anche da h .
- $g \geq h$ vuol dire che ogni esempio (negativo) non classificato da g , non viene classificato nemmeno da h .

Dato che s e g sono entrambe consistenti col training set, lo è anche h .

Candidate elimination algorithm

Candidate Elimination

- $S = \{ \langle \phi \dots \phi \rangle \}; G = \{ \langle ? \dots ? \rangle \}$
 - for each training example e in the training set do
 - { If e is a **positive** example
 - If $h \in S$ is s.t. $h(e)=0$ (i.e., s is not consistent with e)
 - Perform a minimal generalization h of s such that $h(e)=1$ (h is consistent with e)
 - If exists $g \in G$ s.t. not $g \geq h$ remove g from G
 - If e is a **negative** example
 - If $h \in S$ is s.t. $h(e)=1$ (i.e., s is not consistent with e) remove h from S
 - else for each hypothesis $g \in G$ s.t. $g(e)=1$ (i.e., g is not consistent with e)
 - Perform all minimal specializations g' of g such that $g'(e)=0$ (g' is consistent with e)
 - If exists $g \in G$ s.t. not $g \geq h$ remove g from G
- } //end for
- Output S, G

Questo algoritmo considera sia gli esempi negativi che quelli positivi, se non esiste nessun modello congiuntivo allora viene restituito un *VS* vuoto. Se invece, esiste un unico concetto target allora i limiti *S* e *G* convergono in una singola ipotesi dopo che sono state esaminate tutte le istanze del training set. Una cosa importante da notare è che il *VS* è indipendente dall'ordine col quale il training set viene presentato.

Candidate elimination

An Example

The target function is a boolean function

X	Y	Z	C
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0

$S_0 = \{ \langle 0, 0, 0 \rangle \}$
 $G_0 = \{ \langle ?, ?, ? \rangle \}$

$S_1 = \{ \langle 0, 0, 0 \rangle \}$
 $G_1 = \{ \langle ?, ?, ? \rangle \}$

$S_2 = \{ \langle 0, 0, 0 \rangle \}$
 $G_2 = \{ \langle ?, ?, 0 \rangle \}$
 //removed $\langle 1, ?, ? \rangle$ and $\langle ?, 1, ? \rangle$ from G_2

$S_3 = \{ \langle 0, ?, 0 \rangle \}$ **C = not X not Z**
 $G_3 = \{ \langle ?, ?, 0 \rangle \}$ **C = not Z**

- The algorithm provides two conjunctions compatible with the training data
- Since no conjunction does exist in between S_3 and G_3 (G_3 is a minimal generalization of S_3)

$VS = S_3 \cup G_3$

The target function is a boolean function

X	Y	Z	C
1	0	1	0
0	0	0	1
1	1	0	1
1	0	0	1

$S_0 = \{ \langle 0, 0, 0 \rangle \}$
 $G_0 = \{ \langle ?, ?, ? \rangle \}$

$S_1 = \{ \langle 0, 0, 0 \rangle \}$
 $G_1 = \{ \langle 0, ?, ? \rangle, \langle ?, 1, ? \rangle, \langle ?, ? 0 \rangle \}$

$S_2 = \{ \langle 0, 0, 0 \rangle \}$
 $G_2 = \{ \langle 0, ?, ? \rangle, \langle ?, ?, 0 \rangle \}$

$S_3 = \{ \langle ?, ?, 0 \rangle \}$
 $G_3 = \{ \langle ?, ?, 0 \rangle \}$

- The algorithm has converged to $S_3 = G_3$
- The *VS* has a unique conjunctive model **$h = \langle ?, ?, 0 \rangle$**

The target function is a boolean function

X	Y	Z	C
1	0	1	0
0	0	0	1
1	1	0	1
1	0	0	0

$S_0 = \{ \langle 0, 0, 0 \rangle \}$
 $G_0 = \{ \langle ?, ?, ? \rangle \}$

$S_1 = \{ \langle 0, 0, 0 \rangle \}$
 $G_1 = \{ \langle 0, ?, ? \rangle, \langle ?, 1, ? \rangle, \langle ?, ? 0 \rangle \}$

$S_2 = \{ \langle 0, 0, 0 \rangle \}$
 $G_2 = \{ \langle 0, ?, ? \rangle, \langle ?, ?, 0 \rangle \}$

- The version space collapses to the empty set
- Conclusion: No conjunctive hypothesis consistent with the data set exists

$S_3 = \{ \langle ?, ?, 0 \rangle \}$
 $G_3 = \{ \langle ?, ?, 0 \rangle \}$
 $S_4 = \{ \}$
 $G_4 = \{ \langle 0, ?, 0 \rangle, \langle ?, 1, 0 \rangle \} = \{ \}$

Effettuare la classificazione utilizzando il VS

Dato che il VS potrebbe fornire un insieme di modelli validi, anziché uno solo, bisogna decidere quale sia il giusto valore della funzione target.

Unseen instances

Instance	Sky	Temp	Humid	Wind	Water	Forecst	EnjoySpt
A	Sunny	Warm	Normal	Strong	Cool	Change	Yes (100%)
B	Rainy	Cold	Normal	Light	Warm	Same	No (100%)
C	Sunny	Warm	Normal	Light	Warm	Same	yes/ (50%)
D	Sunny	Cold	Normal	Strong	Warm	Same	No (66%)

MODELS

- < sunny, warm, ?, strong, ?, ? >
- < sunny, ?, ?, strong, ?, ? >
- < sunny, warm, ?, ?, ?, ? >
- < ?, warm, ?, strong, ?, ? >
- < sunny, ?, ?, ?, ?, ? >
- < ?, warm, ?, ?, ?, ? >

A: Yes 100% (it is classified by S)

B: No 100% (it is not classified by G)

C: classified by 3 classifiers of the VS out of 6 (50% confidence)

D: classified positive by 2 classifiers out of 6 (33% confidence)

Se una nuova istanza x soddisfa l'ipotesi più specifica nel VS allora essa soddisfa tutte le ipotesi nel VS . Quindi, x è classificata positivamente con una confidenza del 100% (proprio come se la funzione obiettivo fosse conosciuta).

Se una nuova istanza x non soddisfa nessuna delle ipotesi più generali nel VS allora essa è classificata come negativa con una confidenza del 100%.

Se un'istanza x è classificata positiva da qualche ipotesi nel VS essa è classificata come positiva con una confidenza minore del 100% in proporzione a quante ipotesi soddisfa rispetto al numero totale delle ipotesi in VS .

Il non determinismo della fase di classificazione

Utilizzando solo il training data, si possono imparare diverse ipotesi compatibili col training set (cioè $h(x) = f(x)$ per ogni x).

La classificazione utilizza i dati riguardanti il passato per prevedere il futuro, tuttavia nessuno può determinare quale sia l'ipotesi che prevede correttamente il futuro. Questo perché l'induzione non è garanzia per la verità e non è deterministica.

L'assunzione del learning induttivo

Secondo *The Inductive Learning Assumption (ILA)*: qualsiasi ipotesi trovata per rappresentare bene la funzione target su un insieme, di training example, sufficientemente grande, approssimerà bene la funzione target anche sugli esempi non osservati.

2.9 Le ipotesi in DNF

Una cosa importante da capire è quale sia la probabilità che un concetto sia rappresentabile da una sola *coniunzione*.

Supponiamo di avere un training Set S con n attributi booleani. Ci sono:

- $x = 2^n$ possibili istanze;
- $y = 3^n$ ipotesi congiuntive costruibili su S ($\langle ?, 0, \dots, 0 \rangle, \langle 0, ?, \dots, 0 \rangle$);
- $z = 2^x$ concetti che possono essere definiti su S (assumendo una binary classification)

In questo caso, la probabilità è quindi di $\frac{y}{z}$.

Quindi con soli 5 attributi è uguale a $\frac{243}{4.294.967.296}$. Troppo bassa!

Come si può dedurre da questo la probabilità che esista un modello congiuntivo e quasi pari a 0, per questo molto probabilmente esso non esiste, *abbiamo bisogno di un linguaggio più espressivo!*. Per tale motivo potremmo considerare un ipotesi in **Disjunctive normal form (DNF)**.

Un'ipotesi DNF è una disgiunzione di congiunzioni di vincoli sugli attributi. Lo spazio delle ipotesi DNF è **Completo** cioè è capace di rappresentare qualsiasi concetto che può essere imparato.

Esempio:

X	Y	Z	C
0	0	0	0
0	1	0	1
1	1	1	0
1	0	0	1

- $h = \langle 0, 1, 0 \rangle + \langle 1, 0, 0 \rangle$ is a model

Tuttavia, in questo caso ci troviamo di fronte ad un nuovo problema: così facendo, qualsiasi nuova istanza sarà classificata come negativa!

Questo accade perché ***i modelli DNF non sono capaci di generalizzare gli esempi osservati.***

2.9.1 Utilizzate "Candidate Elimination" per indurre ipotesi DNF

Utilizzare il DNF con CE produce dei risultati banali: l'insieme S in questo caso è formato dalla disgiunzione degli esempi positivi osservati, mentre, l'insieme G è formato dalla negazione della disgiunzione degli negativi osservati.

Così facendo, tuttavia, le uniche istanze ad essere classificate con una confidenza del 100% sono esattamente quelli presenti nel training set. Gli altri sono tutte classificate con una confidenza del 50%.

Quindi se viene utilizzato uno spazio DNF, CE non fa altro che indurre dei classificatori banali, a causa dell'inesistente capacità di generalizzazione.

Solo gli esempi nel Training Set saranno classificati senza ambiguità, tutto il resto sarà ambiguo.

Concept Learning Summary

- Concept Learning can be seen as the problem of searching in a given hypothesis space a model
- The “more-general-than” relationship can be exploited for an efficient search
- The FIND-S algorithm finds the most specific conjunction (if any)
- The CANDIDATE-ELIMINATION algorithm computes the Version Space
- Disjunctive models are very unlikely
- DNFs allow to represent any possible subset of examples
- Unfortunately, DNFs have no generalization capabilities!!!

2.9.2 Le ipotesi approssimate

Dato che lo spazio DNF è completo ma CE genera dei modelli DNF banali, abbiamo bisogno di un algoritmo che invece impari dei modelli DNF non banali (se esistono) oppure approssimare meglio le ipotesi DNF.

Quindi se non esiste nessun modello non banale la soluzione migliore è quella di ***approssimare le soluzioni!*** Tuttavia, anche se questo dovesse esistere è preferibile trovare una soluzione approssimata in modo da evitare il problema dell'***overfitting***.

2.9.3 Overfitting (sovradattamento)

Come sappiamo spesso i training data reali potrebbero avere alcuni problemi:

- sono sporchi (noisy), potrebbero infatti contenere delle istanze classificate erroneamente, valori di attributi sbagliati, valori persi ecc.
- potrebbero esserci troppo pochi esempi e quindi essi non sono rappresentativi della funzione target.

In questi casi, ovviamente, un'ipotesi che si adatta perfettamente al training set potrebbe essere una cattiva generalizzazione del concetto che si vuole imparare.

L'overfitting si verifica quando il modello è troppo su misura rispetto ai dati di allenamento in modo che possa riflettere le sue proprietà contingenti piuttosto che le sue proprietà strutturali.

Definizione (by Mitchell): si dice che un'ipotesi h "overfitta" il training data se esiste un'altra ipotesi h' tale che h ha un livello di errore più basso sugli esempi studiati, ma h' ha un livello di errore più basso rispetto agli esempi non visti, cioè ha una capacità di generalizzazione migliore.

Un modello che overfitta il training data non generalizza bene gli esempi nuovi.

Overfitting due to noise

Mammal training data - Both bat and whale are misclassified

Name	Body Temp	Gives birth	4-legged	hibernates	mammal
Porcupine	warm	yes	yes	yes	yes
cat	warm	yes	yes	no	yes
bat	warm	yes	no	yes	no
whale	warm	yes	no	no	no
salamander	cold	no	yes	yes	no
k. dragon	cold	no	yes	no	no
Python	cold	no	no	yes	no
Salmon	cold	no	no	no	no
Eagle	warm	no	no	no	no
Guppy	cold	yes	no	no	no

$h = \langle \text{warm, yes, yes, ?} \rangle$ is a model- a mammal has a warm BT, gives birth, is 4-legged (not correct!!!)

Unseen instances

$h = \langle \text{warm, yes, yes, ?} \rangle$

Name	Body Temp	Gives birth	4-legged	hibernates	mammal	Predicted label
Human	warm	yes	no	no	yes	no
pigeon	warm	no	no	no	no	no
elephant	warm	yes	yes	no	yes	yes
Leopard seal	cold	yes	no	no	no	no
turtle	cold	no	yes	no	no	no
penguin	cold	no	no	no	no	no
eel	cold	no	no	no	no	no
dolphin	warm	yes	no	no	yes	no

- Elephant is correctly classified as mammal
- Both humans and dolphins are misclassified - they are not 4-legged
- All other instances are correctly classified
- 25% test errors

Mammal training data - $h_1 = \langle \text{warm, yes, ?, ?} \rangle$ NOT a model

Name	Body Temp	Gives birth	4-legged	hibernates	mammal
Porcupine	warm	yes	yes	yes	yes
cat	warm	yes	yes	no	yes
bat	warm	yes	no	yes	no
whale	warm	yes	no	no	no
salamander	cold	no	yes	yes	no
k. dragon	cold	no	yes	no	no
Python	cold	no	no	yes	no
Salmon	cold	no	no	no	no
Eagle	warm	no	no	no	no
Guppy	cold	yes	no	no	no

Unseen instances

$h = \langle \text{warm, yes, ?, ?} \rangle$ - 0 test errors

Name	Body Temp	Gives birth	4-legged	hibernates	mammal	Predicted label
Human	warm	yes	no	no	yes	yes
pigeon	warm	no	no	no	no	no
elephant	warm	yes	yes	no	yes	yes
Leopard seal	cold	yes	no	no	no	no
turtle	cold	no	yes	no	no	no
penguin	cold	no	no	no	no	no
eel	cold	no	no	no	no	no
dolphin	warm	yes	no	no	yes	yes

- $h = \langle \text{warm, yes, yes, ?} \rangle$
 - compatible with training data
 - 0% training errors, 25% test errors
- $h_1 = \langle \text{warm, yes, ?, ?} \rangle$
 - NOT compatible with training data
 - 20% training errors, 0% test errors
- h_1 (not a model) performs better than h (model) over unseen data!
- Notice: h_1 is "simpler" than h

Overfitting due to lack of examples

Training set for classifying mammals

name	Body temp	Gives birth	Four-legged	hibernates	mammal
salamander	cold	no	yes	yes	no
Guppy	cold	yes	no	no	no
Eagle	warm	no	no	no	no
Poorwill	warm	no	no	yes	no
Platypus	warm	no	yes	yes	yes

- The only representative mammal is the platypus!!!
- $h = \langle \text{warm, no, yes, yes} \rangle$ is a model
- According to h both humans and elephants are NOT mammals!
- Cause of overfitting: the training set is not statistically representative

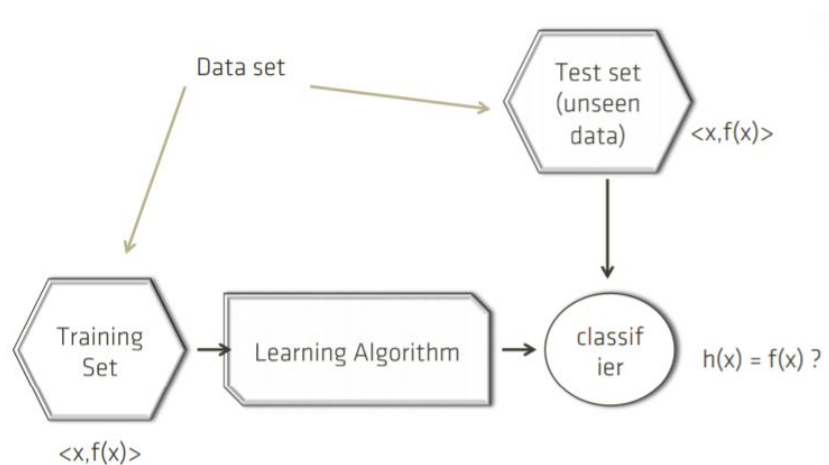
2.10 Classificazione- Valutazione del modello

In molti casi, è importante valutare la performance dell'ipotesi imparata in modo più preciso possibile. Una delle ragioni è semplicemente capire se è il caso di utilizzare l'ipotesi trovata.

Quando viene valutata un'ipotesi si è molto spesso interessati a stimare l'accuratezza con il quale essa classificherà le istanze future. Allo stesso tempo si vorrebbe conoscere la probabilità di errore in questa stima della precisione.

2.10.1 Testing del classificatore

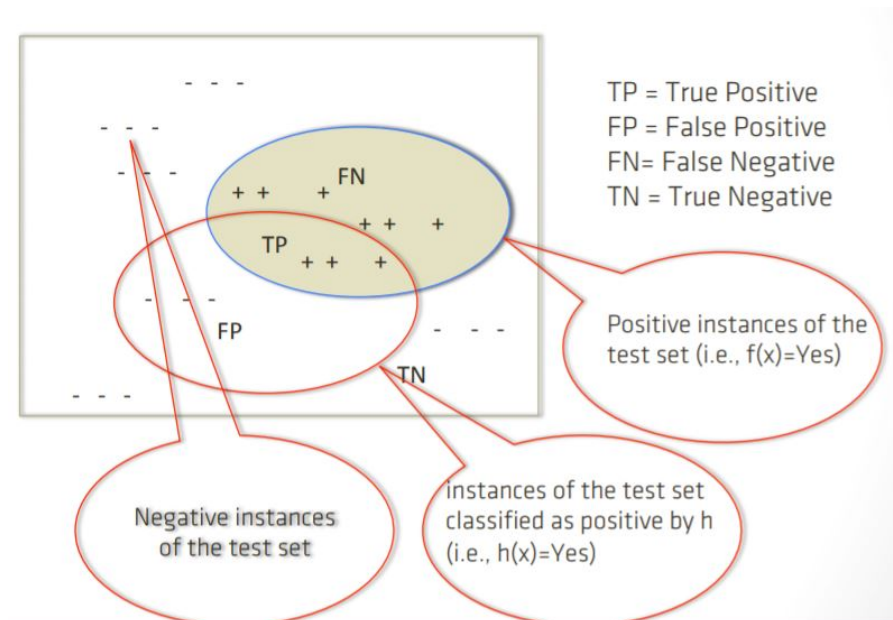
Considerando un algoritmo A e un data Set S di istanze etichettate $\langle x, f(x) \rangle$ l'idea generale del testing del classificatore, per di valutare la capacità di generalizzazione di A , è quella di dividere S in due sottoinsiemi: uno dei quali utilizzato come training e l'altro utilizzato come testing.



Metriche utilizzate per la valutazione delle performance

Come mostra la figura precedente una volta imparata un'ipotesi dal training subset essa viene testata sul testing subset. Durante il testing viene confrontato il risultato ottenuto da essa con la reale classe delle istanze in questo subset. Possiamo ottenere così 4 possibili risultati:

1. **TP**: *True Positive*, si ottiene quando applicando l'ipotesi su un esempio positivo si ottiene comunque un'etichetta positiva. Quindi $f(x) = h(x)$
2. **FP**: *False Positive*, si ottiene quando applicando l'ipotesi su un esempio negativo si ottiene comunque un'etichetta positiva. Quindi $f(x) \neq h(x)$
3. **FN**: *False Negative*, si ottiene quando applicando l'ipotesi su un esempio positivo si ottiene un'etichetta negativa. Quindi $f(x) \neq h(x)$
4. **TN**: *True Negative*, si ottiene quando applicando l'ipotesi su un esempio negativo si ottiene un'etichetta negativa. Quindi $f(x) = h(x)$



METRICHE:

- **Accuratezza**: è data dal numero delle istanze correttamente classificate rispetto al numero totale delle predizioni.

$$Acc = \frac{TP + TN}{N} \text{ dove } N = TP + TN + FN + FP$$

- **Errore**: è dato dal numero delle istanze classificate male sul numero totale delle predizioni.

$$Err = \frac{FP + FN}{N} = 1 - Acc$$

TEST SET								
Sky	Temp	Humid	Wind	Water	Forecst	EnjoySpt	h(x)	
Sunny	Warm	Normal	Strong	Cool	Change	No	No	TN
Rainy	Cold	Normal	Light	Warm	Same	No	No	TN
Sunny	Warm	Normal	Light	Warm	Same	Yes	Yes	TP
Sunny	Cold	Normal	Strong	Warm	Same	Yes	No	FN
Sunny	Warm	high	Strong	Cool	Change	No	No	TN
Rainy	Warm	Normal	Light	Warm	Change	No	No	TN
Rainy	Warm	Normal	Light	Warm	Same	No	Yes	FP
Sunny	Cold	Normal	Strong	Warm	Change	Yes	No	FN

TP = 1

FP = 1

FN = 2

TN = 4

• Acc = 5/8 = 62,5%

• Err = 3/8 = 37,5%

Alcuni problemi:

Supponiamo che la nostra ipotesi h sia un **rejectore** cioè che nessuna istanza venga classificata positiva da essa. Es: $TP = 0, FP = 0, FN = 3, TN = 5$ in questo caso abbiamo un'accuratezza del 62,5%!

A causa di questi casi particolari, la precisione potrebbe non essere una performance measure adeguata, cioè quando il numero dei casi negativi è molto più grande rispetto al numero dei casi positivi.

Supponiamo che abbiamo a disposizione 1000 esempi di cui 995 negativi e 5 positivi. Se l'ipotesi li classifica tutti come negativi. Avremmo che $TP = 0, TN = 995, FP = 0, FN = 5$ dove l'accuratezza sarà del 99,5%, anche se il classificatore ha ignorato interamente tutti i casi positivi!

Per questo motivo viene considerata un'ulteriore metrica **la precisione**. Essa è la frazione delle istanze classificate correttamente,

$$Pr = \frac{TP}{TP + FP}$$

Applicando la precisione all'esempio precedente, si ottiene una precisione dello 0,5 (va da 0 a 1), un numero abbastanza basso rispetto al 62,5% di accuratezza.

Se i false positive sono uguali a zero, allora la precisione sarà di 1, tuttavia, la sola precisione come metrica non è sufficiente poiché un classificatore potrebbe avere un'alta precisione ma una bassa copertura.

Un'altra metrica è la **Recall** che è data da:

$$\frac{TP}{TP + FN}$$

rappresenta quindi la frazione degli esempi positivi nel test set che sono stati correttamente classificati, è chiamata anche **copertura, sensibilità** o **True Positive Rate**.

Applicando la Recall all'esempio precedente, si ottiene 0,33 (va da 0 a 1). Tuttavia anche questa non può essere sufficiente da sola poiché un classificatore potrebbe avere un'alta copertura ma una bassa precisione. Infatti, quando i false negative sono pari a 0 allora il recall sarà pari a 1.

F-measure

Dato che le performance measure precedenti non sono sufficienti se utilizzate da sole, si utilizza la **F-measure** che si ottiene con

$$F = \frac{2PrRe}{Pr + Re}$$

,

$$F \approx \min(Pr, Re)$$

(è approssimativamente uguale al minimo fra i due)

F è alto quando entrambi sia Pr che Re sono alti (ciò definisce un buon classificatore)

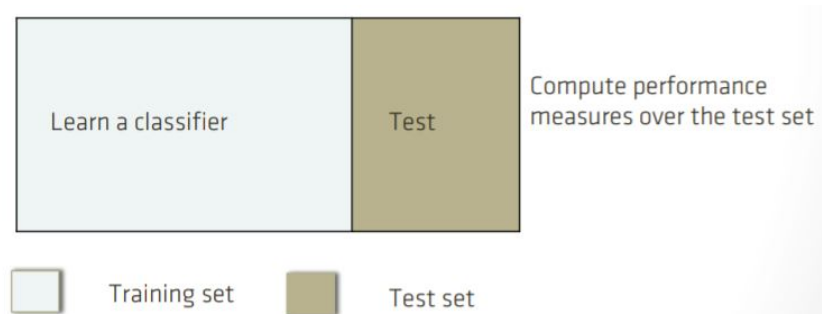
Nell'esempio precedente $F = 0,37$

2.10.2 Tecniche per la valutazione delle performance

Esistono due tecniche utilizzate per verificare la qualità di un classificatore indotto: **Holdout** e **Cross Validation**

2.10.3 Holdout

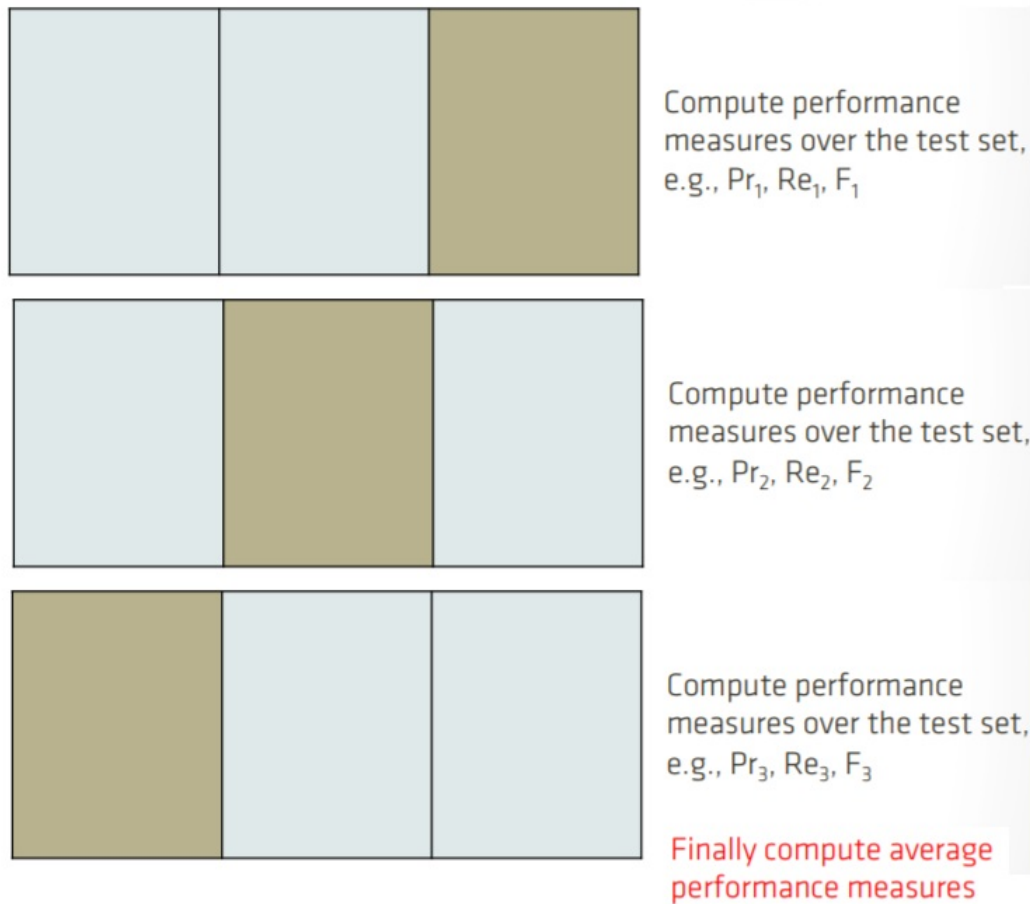
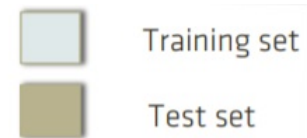
Come nell'esempio precedente viene diviso il training set in due subset: uno verrà utilizzato come training per imparare un concetto, mentre l'altro verrà utilizzato per testare la capacità di generalizzazione dell'ipotesi trovata.



2.11 Cross Validation: K-fold-cross-validation:

Il dataset con questa tecnica viene diviso in k partizioni aventi la stessa dimensione. Durante ogni esecuzione, una delle partizioni viene scelta per il testing mentre le rimanenti $k - 1$ vengono utilizzati per il training. Tale procedura viene ripetuta per k volte. Alla fine viene restituita la media (sulle k partizioni) delle performance measure.

- 3-fold Cross Validation
- 2 folds for learning
- 1 fold for testing



Holdout vs Cross Validation

La valutazione delle performance basate sulla prima tecnica potrebbe essere influenzata dalla scelta di quello che deve essere il training subset oppure il test subset. Mentre la seconda tecnica esplora tutti i dati così viene ottenuta una più precisa performance measure.

Qualità di un classificatore

La qualità di un classificatore viene data da due capacità:

- **Capacità predittiva:** accuratezza, precisione, recall ecc
- **Capacità descrittiva:** cioè l'interpretabilità del modello e la diminuzione della dimensione del classificatore.

Il principio del **rasoio di Occam** migliora sia la capacità predittiva (abbassando il rischio di overfitting) sia la capacità descrittiva.

Secondo questo principio, le entità non dovrebbero essere moltiplicate senza una vera necessità.

Nel caso in cui due modelli hanno la stessa capacità di generalizzazione allora quello più semplice dovrebbe essere preferito questo perché è più semplice lavorare con delle ipotesi semplici piuttosto che con quelle più complesse. Anche perchè le ipotesi semplici si comportano meglio col problema dell'overfitting.

3 Entropy come misura del disordine

Una misura che è comunemente utilizzata è chiamata **entropy** che caratterizza l'(im)purità di un insieme arbitrario di esempi. Essa viene quindi utilizzata come misura del disordine. Essa veniva dapprima utilizzata nella termodinamica ma più recentemente viene utilizzata nella teoria dell'informazione e nel data mining.

I dati uniformemente distribuiti avranno un'alta entropy mentre dei dati distribuiti in modo non uniforme, ma che si concentrano per di più su un singolo valore, allora avranno una bassa entropy.

L'entropy parte da una base statica, il disordine è più probabile dell'ordine. l'informazione è direttamente proporzionale alla sorpresa ma inversamente proporzionale alla probabilità associato.

3.1 L'entropia secondo la teoria dell'informazione

L'informazione di un evento E è proporzionale alla sorpresa generata da E , la quale è inversamente proporzionale alla probabilità che E si verifichi.

Infatti, quando viene osservato un risultato improbabile di un evento, lo associamo con un'alta quantità di informazione. Contrariamente, quando osserviamo un evento molto probabile allora lo assoceremo ad una più piccola quantità di informazione. (essendo molto probabile non ci sorprende il fatto che esso si verifichi, mentre se esso è poco probabile allora il suo verificarsi potrebbe darci delle informazioni nuove e diverse).

Una *variabile random* è una variabile che può assumere un insieme di possibili valori diversi ognuno dei quali è associato ad una probabilità.

Quindi, sia $X = X_1, \dots, X_n$ una variabile random, $p(X_i)$ è la probabilità che il risultato X_i si verifichi.

L'informazione di X_i è inversamente proporzionale a $p(X_i)$ e può essere definita come

$$I(X_i) = \log \frac{1}{p(X_i)} = -\log p(X_i)$$

Più bassa è la probabilità di un evento più alte saranno sia la **sorpresa** che l'**informazione** ad esso associate.

L'**informazione attesa** di $X = X_1, \dots, X_n$, cioè l'informazione media associata ai differenti valori di X , è definita come:

$$\sum p(X_i)I(X_i) = \sum -p(X_i) * \log p(X_i), \text{ con } i = 1, \dots, n$$

la quale è usata anche come definizione di **information entropy**

$$E(X) = \sum -p(X_i) \log p(X_i), \text{ con } i = 1, \dots, n$$

L'entropia di una variabile random è massima quando i risultati hanno tutti la stessa probabilità poiché è massima l'incertezza. Quindi come detto prima, l'entropia è la più alta quando X ha una distribuzione uniforme mentre è la più bassa quando X ha una distribuzione delle probabilità altamente sbilanciata.

L'obiettivo del machine learning è proprio quello di ridurre l'entropia perché per effettuare delle predizioni abbiamo bisogno di eventi predicibili e l'entropia ci dà una misura per quantificare quanto è affidabile una predizione.

3.2 Entropia nel Data Mining

Se utilizziamo l'entropia come misura del grado di disordine di un training set S :

- se tutti gli esempi sono appartenenti alla stessa classe (cioè equivale al massimo ordine) l'entropia $E(S)$ è pari a zero.
- se tutti gli esempi sono ugualmente distribuiti fra le classi, allora l'entropia $E(S)$ è massima.

L'entropia è utilizzata nel data mining anche nei decision trees come criterio per selezionare "il migliore" attributo di divisione.

Più bassa è l'entropia più i dati appartengono alla stessa etichetta e quindi sono omogenei, per questo essa sarebbe in teoria preferibile.

Esempio:

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

- S consists of 14 examples, 9 positive and 5 negative ($[9+, 5-]$)
 - $E(S) = -p_+ \log p_+ - p_- \log p_- = -9/14 \log 9/14 - 5/14 \log 5/14 = 0.94$
- If $S = [7+, 7-]$
 - $E(S) = 1/2 \log 1/2 - 1/2 \log 1/2 = 1$
- If $S = [14+, 0-]$
 - $E(S) = 1 \log 1 - 0 \log 0 = 0$ ($0 \log 0$ is set to 0)

4 Decision tree

Il decision tree learning è un metodo per l'approssimazione delle funzioni target a valori discreti, nel quale la funzione imparata è rappresentata da un decision tree.

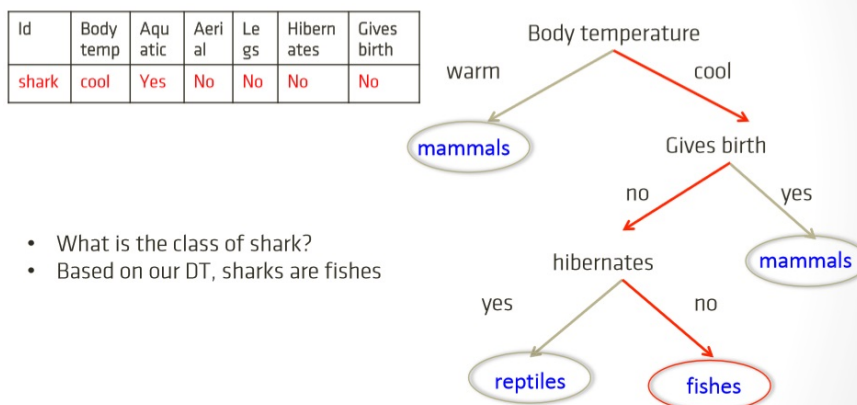
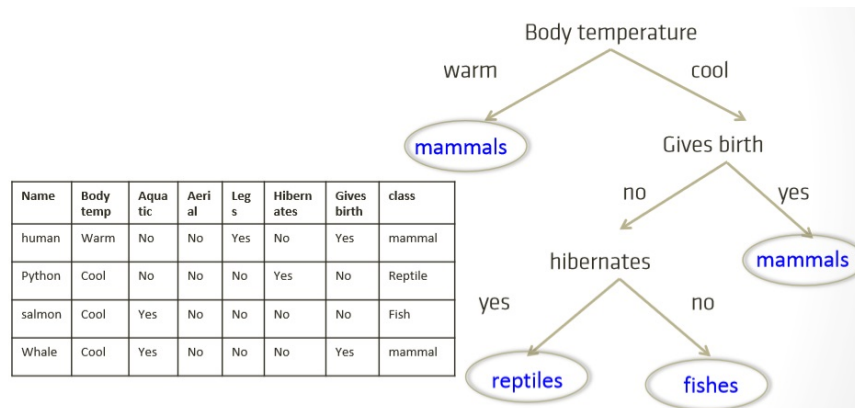
I decision tree classificano le istanze testandole lungo l'albero a partire dal nodo radice fino a qualche nodo foglia, il quale provvede la classificazione dell'istanza.

Ogni nodo nell'albero definisce un test su qualche *attributo* dell'istanza, e ogni ramo uscente da questo nodo corrisponde ad uno dei possibili valori per questo attributo.

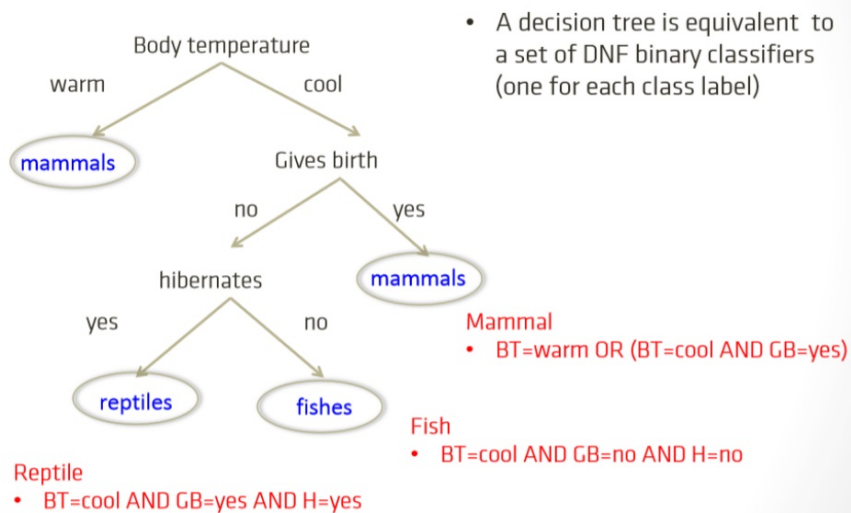
Un'istanza è classificata iniziando dalla radice, testando l'attributo specificato da questo nodo, e successivamente muovendo verso il basso lungo il ramo corrispondente al valore di questo attributo nell'esempio dato. Questo processo è quindi ripetuto per il sottoalbero la cui radice è il questo nuovo nodo (raggiungibile attraverso il ramo corretto).

In generale un decision tree rappresenta una disgiunzione di congiunzioni di constraints sui valori degli attributi delle istanze. Ogni path dalla radice fino ad una foglia corrisponde ad una congiunzione di test sugli attributi e l'albero stesso corrisponde ad una disgiunzione di queste congiunzioni.

Esempio:



- What is the class of shark?
- Based on our DT, sharks are fishes



4.1 DTs il problema del learning

Lo spazio dei decision tree è **completo**, cioè c'è *almeno* un decision tree compatibile col training data. Come detto prima i DT sono in DNF.

Dei decision tree banali possono essere costruiti semplicemente rappresentando ogni esempio con un path dalla root fino alla foglia, inutili tanto quanto le DNF generate dall'algoritmo candidate elimination.

Per tale motivo, vorremmo imparare DTs che non overfittano il training data. Secondo il rasoio di Occam's dei decision tree più piccoli sono preferibili. Quindi l'obiettivo è quello di imparare il più piccolo e non banale Decision tree ottenendo la più piccola quantità di errori nella classificazione del training set. Attualmente questo è un problema appartenente alla classe **NP-hard**.

I decision tree sono generalmente più adatti per problemi:

- *le cui istanze sono rappresentate da coppie attributo-valore.* Cioè descritte da un insieme fissato di attributi e dal loro valore. La più semplice situazione per il decision tree learning è quando ogni attributo può assumere un piccolo numero disgiunto di possibili valori (per esempio: caldo, tiepido, freddo) anche se numerose estensioni dell'algoritmo base consentono anche di gestire bene degli attributi con valori reali (per esempio la temperatura indicata numericamente).
- *La cui funzione target ha dei valori di output discreti.* Infatti, il metodo base esso può essere facilmente esteso per imparare funzioni con più di due possibili valori di output. Inoltre, come per i valori degli attributi discussi al punto precedente, anche per la funzione target una più sostanziale estensione dell'algoritmo base consente di imparare funzioni target con valori reali come output. Anche se bisogna dire che in questo contesto i decision tree non sono generalmente utilizzati.
- *Per i quali possono essere richieste delle descrizioni disgiuntive,* perchè come abbiamo visto i DT rappresentano naturalmente delle espressioni disgiuntive.
- *Il cui training data potrebbe contenere errori o valori mancanti per alcuni attributi.* I DT infatti sono robusti agli errori, sia nelle classificazioni dei training example e sia nei valori degli attributi che descrivono tali esempi. Inoltre, essi possono essere utilizzati anche quando qualche training example ha dei valori non conosciuti per gli attributi.

In genere i DT sono applicati a problemi che sono conosciuti come **problemi di classificazione**.

4.2 L'idea di base Top-Down dei decision tree

L'idea di base di questo metodo è:

1. Scegliere la caratteristica (attributo) C migliore come radice dell'albero;
Siano $v_1, \dots, v_k$ i possibili valori di C all'interno del training set
2. Dividere il training set S in sottoinsiemi $S_{v_1}, \dots, S_{v_k}$ dove ogni sottoinsieme S_{v_i} contiene tutti gli esempi che hanno lo stesso valore v_i per C .

3. Applicare ricorsivamente l'algoritmo ad ogni nuovo sottoinsieme S_{v_i} finché tutti gli esempi non hanno la stessa class label.

4.3 Algoritmo ID3

Basic DT Algorithm ID3

ID3(Examples S, Attributes Att)

create a Root node;

/* exit conditions

If S contains only one class label C

return the single-node tree Root labeled C

If Att empty

return the single-node tree Root with label= most frequent class in S

/* recursive step

Select an attribute $A \in \text{Att}$; **/* a is non-deterministically chosen**

• label Root by A; ;

for each value v of A:

• Add a new edge E below Root labeled by v;

• Let S_v be the subset of S where $A=v$;

• If S_v is empty then add a leaf node to E with label= most frequent class in S

• Else add the subtree ID3 (S_v , Att - {A}) to E

The vertebrate example

ID3(Examples S, Attributes Att)
Create a Root node

/* Recursive step
Select an attribute $A \in \text{Att}$; **/* A = BT**
• label Root by "BT";

Name	Body temp	Aquatic	Aerial	Legs	Hibernates	Gives birth	class
human	Warm	No	No	Yes	No	Yes	mammal
Python	Cool	No	No	No	Yes	No	Reptile
salmon	Cool	Yes	No	No	No	No	Fish
Whale	Cool	Yes	No	No	No	Yes	mammal



ID3(Examples S, Attributes Att)
Create a Root node

/* Recursive step
Select an attribute $A \in \text{Att}$; **/* A = BT**
• label Root by "BT";
for each value v in {warm, cool} of "BT"

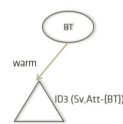
/* Select v= warm

1. Add a new arc below Root with label "warm";

2. S_v = set of examples with BT= warm;

3. Below the new arc create the subtree ID3 (S_v , Att-{BT})

Name	Body temp	Aquatic	Aerial	Legs	Hibernates	Gives birth	class
human	Warm	No	No	Yes	No	Yes	mammal



Set S_v of examples with BT= warm

ID3(Examples S, Attributes A)
Create a Root node

/* Exit condition 1 occurs

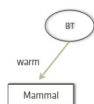
If S contains only one class **/* Class = mammal**

return the single-node tree Root labeled mammal

...

Name	Body temp	Aquatic	Aerial	Legs	Hibernates	Gives birth	class
human	Warm	No	No	Yes	No	Yes	mammal

Set S_v of examples with BT= warm



ID3(Examples S, Attributes A)

/* recursive step

/* attribute a= Italian selected

for each value v in {warm, cool} of "BT"

/* Select v= cool

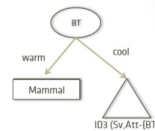
1. Add a new arc below Root with label "cool";

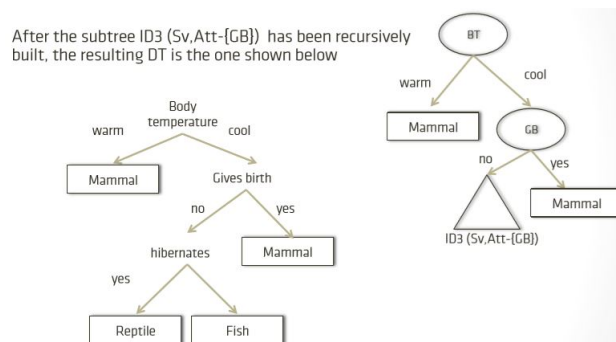
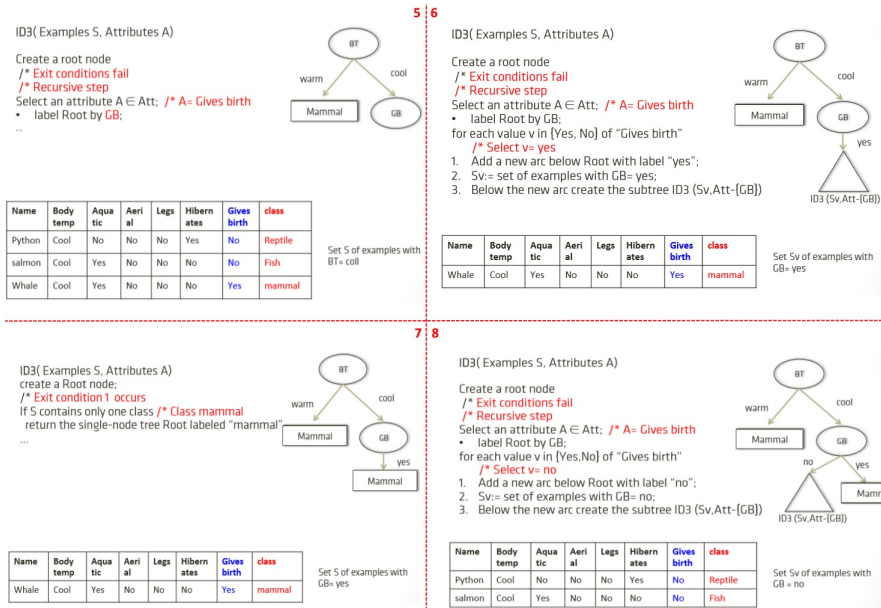
2. S_v = set of examples with BT= cool;

3. Below the new arc create the subtree ID3 (S_v , Att-{BT})

Name	Body temp	Aquatic	Aerial	Legs	Hibernates	Gives birth	class
Python	Cool	No	No	No	Yes	No	Reptile
salmon	Cool	Yes	No	No	No	No	Fish
Whale	Cool	Yes	No	No	No	Yes	mammal

Set S_v of examples with BT= cool





Se l'insieme S_v è vuoto allora viene aggiunta una foglia ad un arco la cui etichetta sarà uguale a quella della classe che ha la frequenza più alta.

4.4 L'algoritmo ID3 base

ID3 base estrae un attributo in **modo non deterministico**.

Ci sono tre condizioni che definiscono la terminazione dell'algoritmo, per un nodo foglia N l'algoritmo termina se gli esempi associati con N hanno tutti la stessa classe target, se non ci sono più attributi con il quale splittare gli esempi e infine se il sottoinsieme S_v del training set S è vuoto. Tuttavia, se queste ultime due condizioni si verificano allora vuol dire che quel DT selezionato potrebbe non essere un modello del training data.

Basic DT Algorithm ID3

```
ID3( Examples S, Attributes Att)
create a Root node;
/* exit conditions
If S contains only one class label C
    return the single-node tree Root labeled C
If Att empty
    return the single-node tree Root with label= most frequent class in S
/* recursive step
Select the best attribute A ∈ Att; /* what is the best attribute?
• label Root by A; ;
for each value v of A:
• Add a new edge E below Root labeled by v;
• Let Sv be the subset of S where A=v;
• If Sv is empty then add a leaf node to E with label= most frequent
  class in S
• Else add the subtree ID3 (Sv, Att - {A}) to E
```

ID3 sceglie l'attributo che meglio divide il training data poiché la scelta degli attributi deve essere guidata dall'obiettivo di creare un DT piccolo.

Per fare tale scelta utilizza la nozione di **Information Gain** la quale è basata sulla nozione di **Entropy** discussa precedentemente. Infatti, l'entropia può essere utilizzata come misura del disordine di un training data. Se tutti gli esempi sono nella stessa classe allora l'entropia è pari a 0, al contrario, se tutti gli esempi sono equamente distribuiti sulle classi l'entropia è massima.

4.5 L'Information Gain

L'**Information Gain** $IG(S, A)$ è la riduzione dell'entropia aspettata dal partizionamento degli esempi S secondo un attributo A. E può essere definita come:

$$IG(S, A) = E(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} E(S_v)$$

dove S_v è il sottoinsieme S dove $A = v$. E' importante notare che $IG(S, A)$ è massimo quando $E(S_v) = 0$, per ogni v.

Information Gain - Example

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

- The attribute Wind has the maximum discriminating power
 - If Wind= weak then Class=yes
 - If Wind= strong then Class=No
- The IG, i.e., the reduction of entropy, is indeed maximum

- $S = [9+, 5-], |S| = 14$
- $E(S) = - 9/14 \log 9/14 - 5/14 \log 5/14 = 0.94$
- $A = \text{wind}, \text{Values}(A) = \{\text{weak}, \text{strong}\}$
- $S_{\text{weak}} = [6+, 2-], S_{\text{strong}} = [3+, 3-]$
- $|S_{\text{weak}}| = 8, |S_{\text{strong}}| = 6$
- $E(S_{\text{weak}}) = - 6/8 \log 6/8 - 2/8 \log 2/8 = 0.81$
- $E(S_{\text{strong}}) = - 3/6 \log 3/6 - 3/6 \log 3/6 = 1.00$
- $IG(S, \text{wind}) = E(S) - |S_{\text{weak}}| / |S| * E(S_{\text{weak}}) - |S_{\text{strong}}| / |S| * E(S_{\text{strong}})$
 $= 0.94 - 8/14 * 0.81 - 6/14 * 1 = 0.05$
- $A = \text{wind}, \text{Values}(A) = \{\text{weak}, \text{strong}\}$
- $S_{\text{weak}} = [8+, 0-], S_{\text{strong}} = [0+, 6-]$
- $E(S_{\text{weak}}) = 0$
- $E(S_{\text{strong}}) = 0$
- $IG(S, \text{wind}) = E(S) - 0 - 0 = 0.94$

l'information Gain $IG(S, A)$ è la riduzione aspettata che la partizione degli esempi di S rispetto ad A causa all'entropia. $IG(S, A)$ è massima quando gli esempi all'interno di ogni sottoinsieme del training set, creati dalla partizione di S attraverso A , sono tutti etichettati con la stessa etichetta. Mentre essa è minima (i.e. 0) quando ogni sottoinsieme ha la stessa distribuzione della class label di A . Più A è discriminante, più alta è l'IG. Quindi ogni volta viene scelto l'attributo con la più alta IG

Basic DT Algorithm ID3 (Based on IG)

ID3(Examples S , Attributes Att)

create a Root node;

/* exit conditions

If S contains only one class label C

return the single-node tree Root labeled C

If Att empty

return the single-node tree Root with label= most frequent class in S

/* recursive step

Select the attribute $A \in Att$ s.t. $IG(S, A)$ is maximum

• label Root by A ; ;

for each value v of A :

• Add a new edge E below Root labeled by v ;

• Let S_v be the subset of S where $A=v$;

• If S_v is empty then add a leaf node to E with label= most frequent class in S

• Else add the subtree ID3 ($S_v, Att - \{A\}$) to E

come nodo del sotto albero.

Sommario ID3

Questo algoritmo è basato su una strategia greedy, general to specific e a partizionamento ricorsivo. Ad ogni step, ID3 seleziona l'attributo che meglio separa gli esempi dati attraverso l'uso dell'IG. In questo modo, viene generato un DT piccolo e bilanciato. La strategia di dividere il training set basandosi su un approccio statico consente agli algoritmi di learning di trovare di DT accurati senza overfittare il training data. L'output però non è necessariamente consistente col training data poichè l'algoritmo non torna mai indietro per riconsiderare le scelte già effettuate precedentemente.

4.6 L'indice GINI

Come l'Entropia l'indice Gini è una misura per l'impurità definita come

$$Gini(S) = 1 - \sum_c p(c)^2$$

$$Gain(A, S) = Gini(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} Gini(S_v)$$

Se l'entropia è utilizzata al posto dell'indice Gini allora $Gain = IG$

4.7 Attributi continui

L'algoritmo ID3 è progettato per lavorare con degli attributi **Categorici**, quindi bisogna estendere il suo raggio agli attributi numerici per gestire meglio un possibile scenario reale.

Un modo potrebbe essere quello di definire dei nuovi attributi a valori discreti che partizionano in più intervalli gli attributi che contengono i valori interi, ognuno dei quali viene trattato come un attributo categorico. Un altro approccio potrebbe essere quello di trovare una *soglia* in modo da creare dei nodi binari del tipo $X < s$ oppure $X \geq t$. Questa soglia può essere calcolata ordinando dapprima gli esempi secondo l'attributo X, successivamente spostare la soglia s a partire dal valore più piccolo verso quello più grande. Infine bisogna selezionare la s che dà la migliore information gain.

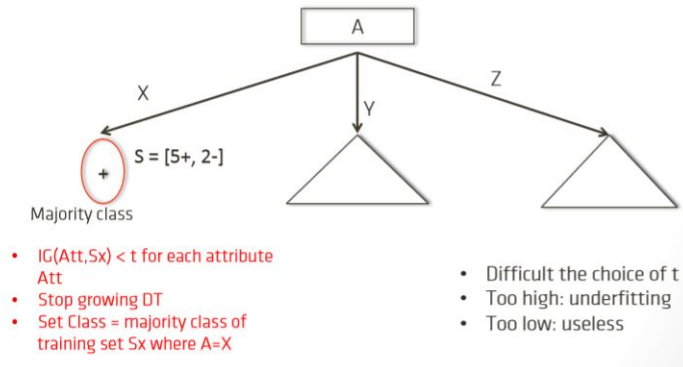
4.7.1 Overfitting

Dei piccoli DT rappresentano delle più generali DNF quindi essi sono meno portati ad overfit il training data. Infatti, imparare un albero che si adatta troppo agli esempi disponibili non porta ad imparare l'albero che invece meglio si comporta con la generalizzazione per i dati non visti.

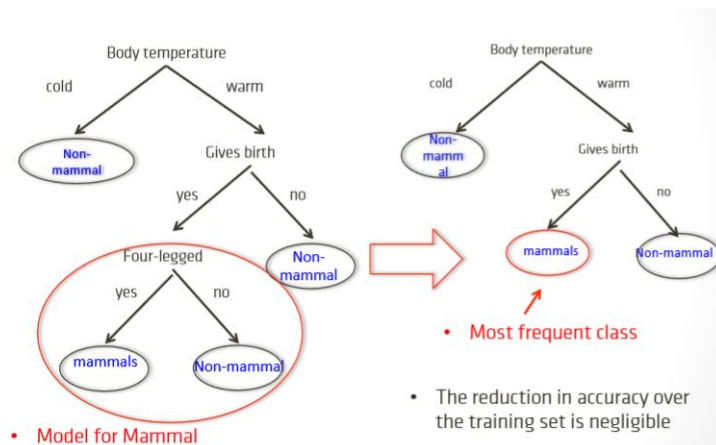
4.8 Ridurre la dimensione di un DT

In modo da evitare di generare degli alberi troppo grandi che rischiano di overfittare i dati disponibili si applicano alcune tecniche: **Pre-Pruning**: l'algoritmo per la generazione dell'albero viene fermato prima di generare l'albero pieno. A tal proposito vengono utilizzate delle nuove condizioni che consentono di capire quando fermare il

processo. Per esempio: l'algoritmo si deve fermare se l'information gain di un nodo foglia è più piccolo di una determinata soglia. **Post-Pruning:** In questo caso, in-



vece, viene generato l'intero albero fino alla dimensione massima, successivamente, viene sostituito un sotto albero da un singolo nodo se il livello di errore non peggiora *significativamente*. Questo processo viene eseguito finché ci sono sottoalberi idonei alla potatura.



4.9 Estensioni di ID3

ID3 è la prima implementazione dei Decision Tree, tuttavia negli anni esso è stato esteso in vari modi, da altri algoritmi, ad esempio:

- **C4.5:**Quinlan's next iteration, questa estensione accetta sia attributi continui che discreti e riesce pure a lavorare con dei dati incompleti. Inoltre, risolve il problema dell'overfitting attraverso una tecnica di post-pruning *molto intelligente*.
- **CART**, o Classification And Regression Trees. Questo approccio costruisce un albero binario, dove ogni nodo interno ha esattamente due rami uscenti. Successivamente l'albero ottenuto è potato attraverso una cost-complexity Pruning. Esso può lavorare sia con variabili categoriche che numeriche e può gestire semplicemente gli errori.